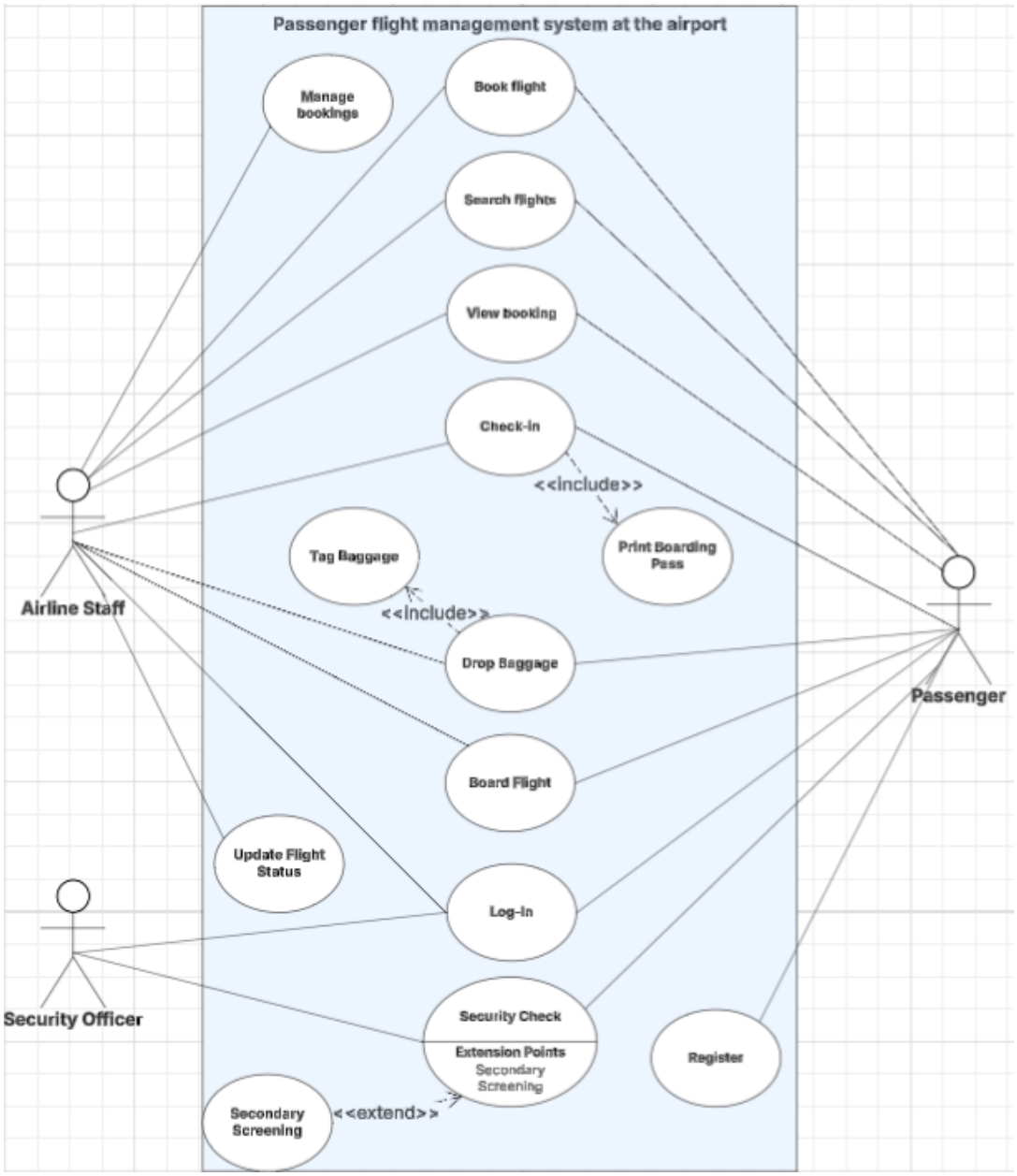
AIRPORT MANAGMENT SYSTEM

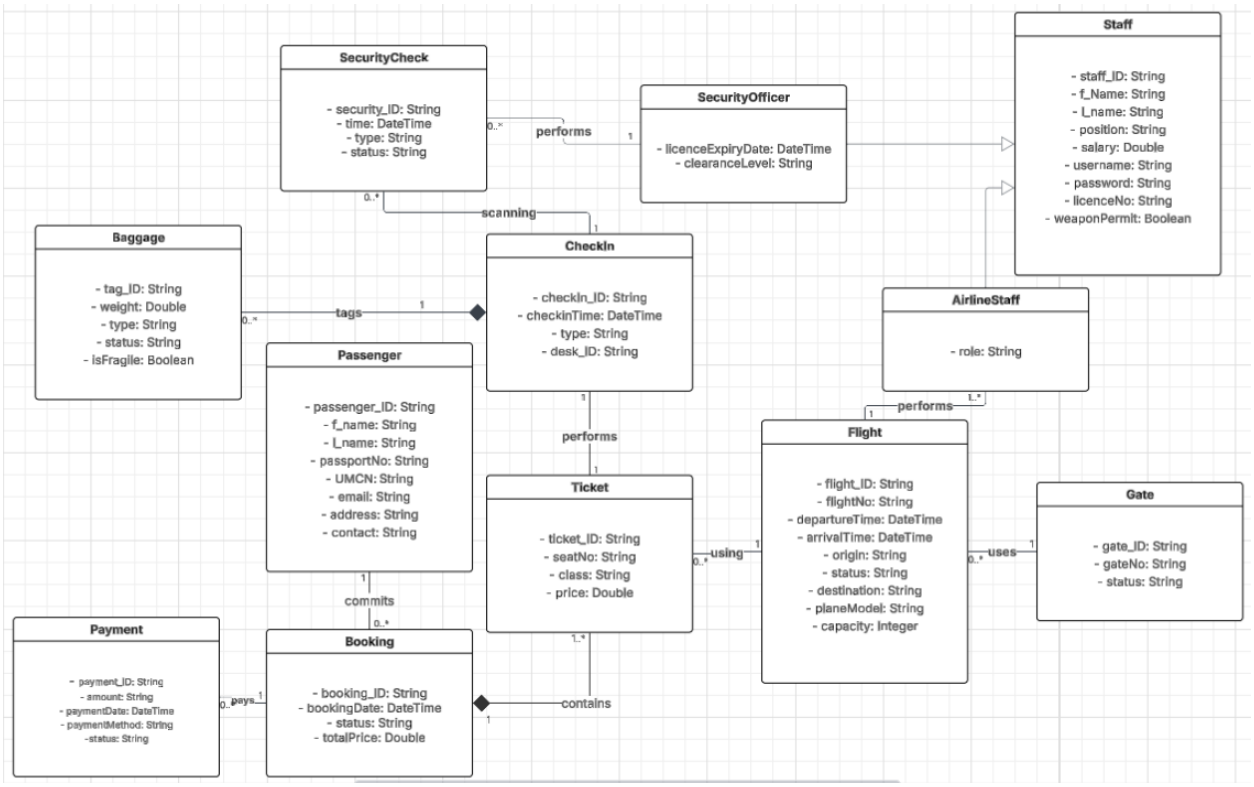
Mateja Glišović

IT 70/2024

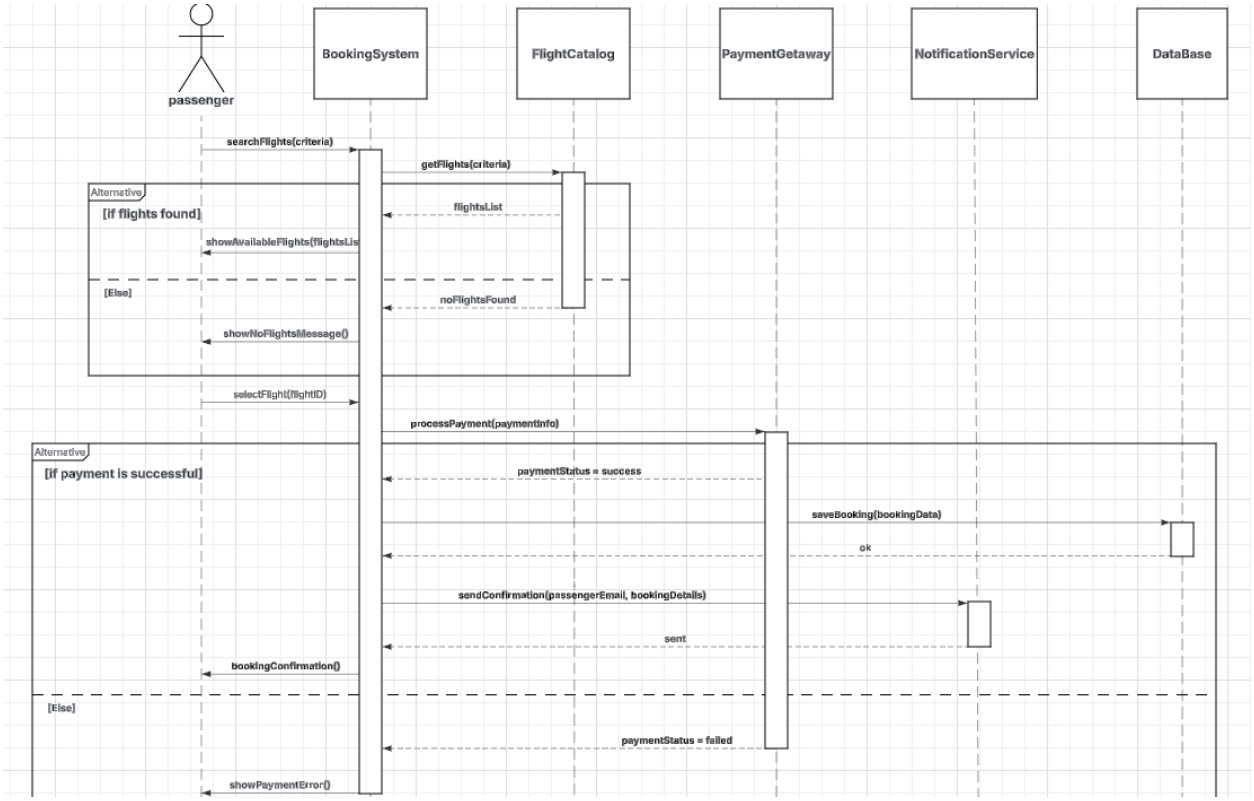
1. UML Dijagram



2. Class Dijagram



3. Squence Dijagram



1. Tekstualni opisi slucaja upotrebe

1.1 Slučaj upotrebe: Prijava na sistem (Logovanje)

- **Kratak opis:** Korisnik (Zaposleni ili Putnik) unosi svoje pristupne podatke kako bi dobio pristup odgovarajućem delu aplikacije. Sistem na osnovu unetih podataka prepoznaje ulogu korisnika i preusmerava ga na Admin Panel ili Putnički Dashboard.
- **Učesnici:** Airline Staff (Administrator), Putnik.
- **Uslovi koji moraju biti zadovoljeni pre:** Aplikacija je pokrenuta i postoji konekcija sa bazom podataka. Korisnik mora biti prethodno registrovan u sistemu.
- **Opis:** Korisnik u predviđena polja unosi svoje korisničko ime i lozinku. Klikom na dugme "Login", sistem vrši proveru u tabelama `tbl_Staff` i `tbl_Passenger`. Ukoliko se podaci poklapaju sa bazom, korisnik dobija pristup.
- **Izuzetak:** Korisnik unosi pogrešnu lozinku ili nepostojeće korisničko ime. Sistem tada prikazuje poruku o grešci "Invalid username or password" i ne dozvoljava pristup.
- **Uslovi koji moraju biti zadovoljeni nakon:** Korisnik je uspešno ulogovan i otvoren je odgovarajući prozor (MainWindow za osoblje ili PassengerDashboard za putnike).

1.2 Slučaj upotrebe: Upravljanje podacima o putnicima

- **Kratak opis:** Zaposleni ima ovlašćenje da unosi nove putnike u sistem, menja njihove podatke ili ih briše.
- **Učesnici:** Airline Staff.
- **Uslovi koji moraju biti zadovoljeni pre:** Zaposleni mora biti ulogovan u sistem.
- **Opis:** Zaposleni u tabu "Passengers" bira opciju za dodavanje novog putnika. Unosi lične podatke (Ime, Prezime, Broj pasoša, JMBG, Kontakt). Sistem automatski generiše jedinstveni ID za putnika. Takođe, zaposleni može izmeniti podatke postojećeg putnika ukoliko dođe do promene (npr. promena adrese ili broja telefona).
- **Izuzetak:** Prilikom brisanja putnika, ukoliko putnik ima aktivne rezervacije, sistem prvo mora obrisati sve vezane podatke (karte, check-in zapise) kako bi očuvao integritet baze, pre nego što obriše samog putnika.
- **Uslovi koji moraju biti zadovoljeni nakon:** Podaci o putniku su trajno sačuvani ili ažurirani u bazi podataka.

1. 3 Slučaj upotrebe: Kreiranje nove rezervacije (Book Flight)

- **Kratak opis:** Zaposleni kreira rezervaciju leta za određenog putnika, definišući sve parametre leta i karte.
- **Učesnici:** Airline Staff.
- **Uslovi koji moraju biti zadovoljeni pre:** Putnik mora biti registrovan u sistemu. Let mora postojati u redu letenja.
- **Opis:** Zaposleni bira putnika iz liste i željeni let. Zatim definiše klasu (Economy, Business, First), broj sedišta i cenu karte. Sistem takođe omogućava odabir tipa prijave na let (Live ili Online). Klikom na dugme "Create", sistem kreira zapis u tabeli rezervacija (`tbl_Booking`) i tabeli karata (`tbl_Ticket`).
- **Izuzetak:** Sistem vrši proveru duplikata. Ukoliko zaposleni pokuša da kreira rezervaciju za putnika koji već ima rezervisan isti let, sistem prikazuje upozorenje "This passenger already has a reservation for this flight" i sprečava upis u bazu.
- **Uslovi koji moraju biti zadovoljeni nakon:** Rezervacija je kreirana sa statusom "Confirmed" ili "Pending", karta je izdata, i generisan je Check-In zapis.

1.4 Slučaj upotrebe: Otkazivanje rezervacije

- **Kratak opis:** Zaposleni briše postojeću rezervaciju iz sistema na zahtev putnika ili usled drugih okolnosti.
- **Učesnici:** Airline Staff.
- **Uslovi koji moraju biti zadovoljeni pre:** Rezervacija mora postojati u sistemu.
- **Opis:** Zaposleni u tabu "Reservations" selektuje željenu rezervaciju i bira opciju za brisanje. Sistem traži potvrdu akcije. Nakon potvrde, sistem kaskadno briše sve povezane podatke: prvo Check-In zapis, zatim kartu, podatke o plaćanju, i na kraju samu rezervaciju.
- **Uslovi koji moraju biti zadovoljeni nakon:** Svi podaci vezani za tu rezervaciju su trajno uklonjeni iz sistema, a mesto na letu je oslobođeno.

1.5 Slučaj upotrebe: Pregled letova (Passenger Dashboard)

- **Kratak opis:** Putnik pristupa sistemu kako bi video listu svojih letova i njihov trenutni status.
- **Učesnici:** Putnik.
- **Uslovi koji moraju biti zadovoljeni pre:** Putnik mora biti ulogovan.
- **Opis:** Aplikacija automatski učitava sve letove povezane sa ID-jem ulogovanog putnika. Prikazuju se informacije o broju leta, destinaciji, vremenu polaska, sedištu i statusu rezervacije (Pending/Confirmed).
- **Uslovi koji moraju biti zadovoljeni nakon:** Putnik ima uvid u svoje rezervacije.

1.6 Slučaj upotrebe: Potvrda i plaćanje rezervacije

- **Kratak opis:** Putnik potvrđuje rezervaciju koja je bila na čekanju (Pending) i vrši simulaciju plaćanja.
- **Učesnici:** Putnik.

- **Uslovi koji moraju biti zadovoljeni pre:** Putnik mora imati rezervaciju sa statusom "Pending".
- **Opis:** Putnik selektuje let koji je označen kao "Pending". Sistem aktivira dugme za potvrdu. Putnik mora štiklirati polje "I agree to Terms of Service". Klikom na dugme "Confirm & Pay", sistem menja status rezervacije u "Confirmed" i evidentira uplatu u bazi podataka.
- **Izuzetak:** Ukoliko putnik pokuša da potvrdi rezervaciju bez prihvatanja uslova korišćenja, sistem prikazuje upozorenje i ne dozvoljava nastavak procesa.
- **Uslovi koji moraju biti zadovoljeni nakon:** Status rezervacije je promenjen u "Confirmed" i putnik više ne može ponovo izvršiti plaćanje za isti let.

2. Dizajn i Implementacija Baze Podataka

2.1 Uvod u arhitekturu podataka

Za potrebe realizacije informacionog sistema za upravljanje aerodromskim saobraćajem ("Airport Management System"), kao

sistem za upravljanje bazom podataka (RDBMS) odabran je **Microsoft SQL Server**. Izbor ove tehnologije zasnovan je na njenoj pouzdanosti, skalabilnosti i robusnoj podršci za transakcione operacije, što je od ključnog značaja za sisteme gde je integritet podataka (poput rezervacija i finansijskih transakcija) prioritet.

Baza podataka, pod nazivom `AirportSystem`, dizajnirana je u skladu sa principima **Relacionog modela podataka**. Osnovni cilj prilikom projektovanja bio je postizanje visoke efikasnosti u pretraživanju podataka i osiguranje konzistentnosti kroz primenu ograničenja (Constraints) i stranih ključeva (Foreign Keys).

```
/****** Object:  Table [dbo].[tbl_Baggage]    Script Date: 14.12.2025. 14:47:29 ******/
SET ANSI_NULLS ON
GO
SET QUOTED_IDENTIFIER ON
GO
CREATE TABLE [dbo].[tbl_Baggage](
    [baggabe_ID] [int] IDENTITY(1,1) NOT NULL,
    [weight] [decimal](5, 2) NOT NULL,
    [type] [nvarchar](20) NOT NULL,
    [isFragile] [bit] NOT NULL,
    [status] [nvarchar](15) NOT NULL,
    [checkIn_ID] [int] NOT NULL,
 CONSTRAINT [PK_tbl_Baggage] PRIMARY KEY CLUSTERED
(
    [baggabe_ID] ASC
)WITH (PAD_INDEX = OFF, STATISTICS_NORECOMPUTE = OFF, IGNORE_DUP_KEY = OFF, ALLOW_ROW_LOCKS = ON, ALLOW_PAGE_LOCKS = ON, OPTIMIZE_FOR_SEQUENTIA
) ON [PRIMARY]
GO
/****** Object:  Table [dbo].[tbl_Booking]    Script Date: 14.12.2025. 14:47:29 ******/
SET ANSI_NULLS ON
GO
SET QUOTED_IDENTIFIER ON
GO
CREATE TABLE [dbo].[tbl_Booking](
    [booking_ID] [int] IDENTITY(1,1) NOT NULL,
    [bookingDate] [datetime] NOT NULL,
    [status] [nvarchar](50) NOT NULL,
    [totalPrice] [decimal](5, 2) NOT NULL,
    [passenger_ID] [int] NOT NULL,
 CONSTRAINT [PK_tbl_Booking] PRIMARY KEY CLUSTERED
(
    [booking_ID] ASC
)WITH (PAD_INDEX = OFF, STATISTICS_NORECOMPUTE = OFF, IGNORE_DUP_KEY = OFF, ALLOW_ROW_LOCKS = ON, ALLOW_PAGE_LOCKS = ON, OPTIMIZE_FOR_SEQUENTIA
) ON [PRIMARY]
```

2.1 tbl_Baggage & tbl_Booking

```
/****** Object:  Table [dbo].[tbl_Flight]    Script Date: 14.12.2025. 14:47:29 ******/
SET ANSI_NULLS ON
GO
SET QUOTED_IDENTIFIER ON
GO
CREATE TABLE [dbo].[tbl_Flight](
    [flight_ID] [int] IDENTITY(1,1) NOT NULL,
    [flightNo] [nvarchar](20) NOT NULL,
    [destination] [nvarchar](50) NOT NULL,
    [departureTime] [datetime] NOT NULL,
    [arrivalTime] [datetime] NOT NULL,
    [status] [nvarchar](20) NOT NULL,
    [gate_ID] [int] NULL,
    [airlineStaff_ID] [int] NULL,
 CONSTRAINT [PK_tbl_Flight] PRIMARY KEY CLUSTERED
(
    [flight_ID] ASC
)WITH (PAD_INDEX = OFF, STATISTICS_NORECOMPUTE = OFF, IGNORE_DUP_KEY = OFF, ALLOW_ROW_LOCKS = ON, ALLOW_PAGE_LOCKS = ON, OPTIMIZE_FOR_SEQUENTIA
) ON [PRIMARY]
GO
/****** Object:  Table [dbo].[tbl_Gate]    Script Date: 14.12.2025. 14:47:29 ******/
SET ANSI_NULLS ON
GO
SET QUOTED_IDENTIFIER ON
GO
CREATE TABLE [dbo].[tbl_Gate](
    [gate_ID] [int] IDENTITY(1,1) NOT NULL,
    [gateNo] [nvarchar](10) NOT NULL,
    [status] [nvarchar](20) NOT NULL,
 CONSTRAINT [PK_tbl_Gate] PRIMARY KEY CLUSTERED
(
    [gate_ID] ASC
)WITH (PAD_INDEX = OFF, STATISTICS_NORECOMPUTE = OFF, IGNORE_DUP_KEY = OFF, ALLOW_ROW_LOCKS = ON, ALLOW_PAGE_LOCKS = ON, OPTIMIZE_FOR_SEQUENTIA
) ON [PRIMARY]
GO
```

2.2 tbl_Flight & tbl_Gate

```
/****** Object:  Table [dbo].[tbl_Passenger]    Script Date: 14.12.2025. 14:47:29 ******/
SET ANSI_NULLS ON
GO
SET QUOTED_IDENTIFIER ON
GO
CREATE TABLE [dbo].[tbl_Passenger](
    [passenger_ID] [int] IDENTITY(1,1) NOT NULL,
    [f_name] [nvarchar](20) NOT NULL,
    [l_name] [nvarchar](20) NOT NULL,
    [passportNo] [nvarchar](20) NOT NULL,
    [UMCN] [nchar](13) NOT NULL,
    [email] [nvarchar](100) NOT NULL,
    [address] [nvarchar](30) NOT NULL,
    [contact] [nvarchar](20) NOT NULL,
    [username] [nvarchar](50) NOT NULL,
    [password] [nvarchar](50) NOT NULL,
 CONSTRAINT [PK_tbl_Passenger] PRIMARY KEY CLUSTERED
(
    [passenger_ID] ASC
)WITH (PAD_INDEX = OFF, STATISTICS_NORECOMPUTE = OFF, IGNORE_DUP_KEY = OFF, ALLOW_ROW_LOCKS = ON, ALLOW_PAGE_LOCKS = ON, OPTIMIZE_FOR
 CONSTRAINT [IX_tbl_Passenger] UNIQUE NONCLUSTERED
(
    [username] ASC
)WITH (PAD_INDEX = OFF, STATISTICS_NORECOMPUTE = OFF, IGNORE_DUP_KEY = OFF, ALLOW_ROW_LOCKS = ON, ALLOW_PAGE_LOCKS = ON, OPTIMIZE_FOR
 CONSTRAINT [IX_tbl_Passenger_1] UNIQUE NONCLUSTERED
(
    [email] ASC
)WITH (PAD_INDEX = OFF, STATISTICS_NORECOMPUTE = OFF, IGNORE_DUP_KEY = OFF, ALLOW_ROW_LOCKS = ON, ALLOW_PAGE_LOCKS = ON, OPTIMIZE_FOR
) ON [PRIMARY]
GO
```

2.3 tbl_Passenger

```
CREATE TABLE [dbo].[tbl_Payment](
    [payment_ID] [int] IDENTITY(1,1) NOT NULL,
    [amount] [decimal](5, 2) NOT NULL,
    [paymentDate] [datetime] NOT NULL,
    [paymentMethod] [nvarchar](50) NOT NULL,
    [status] [nvarchar](20) NOT NULL,
    [booking_ID] [int] NOT NULL,
 CONSTRAINT [PK_tbl_Payment] PRIMARY KEY CLUSTERED
(
    [payment_ID] ASC
)WITH (PAD_INDEX = OFF, STATISTICS_NORECOMPUTE = OFF, IGNORE_DUP_KEY = OFF, ALLOW_ROW_LOCKS = ON, ALLOW_PAGE_LOCKS = ON, OPTIMIZE_FOR_SEQUENTIA
) ON [PRIMARY]
GO
/****** Object:  Table [dbo].[tbl_SecurityCheck]    Script Date: 14.12.2025. 14:47:29 ******/
SET ANSI_NULLS ON
GO
SET QUOTED_IDENTIFIER ON
GO
CREATE TABLE [dbo].[tbl_SecurityCheck](
    [security_ID] [int] IDENTITY(1,1) NOT NULL,
    [securityTime] [datetime] NOT NULL,
    [type] [nvarchar](20) NOT NULL,
    [status] [nvarchar](20) NOT NULL,
    [checkIn_ID] [int] NOT NULL,
    [staff_ID] [int] NOT NULL,
 CONSTRAINT [PK_tbl_SecurityCheck] PRIMARY KEY CLUSTERED
(
    [security_ID] ASC
)WITH (PAD_INDEX = OFF, STATISTICS_NORECOMPUTE = OFF, IGNORE_DUP_KEY = OFF, ALLOW_ROW_LOCKS = ON, ALLOW_PAGE_LOCKS = ON, OPTIMIZE_FOR_SEQUENTIA
) ON [PRIMARY]
GO
/****** Object:  Table [dbo].[tbl_Staff]    Script Date: 14.12.2025. 14:47:29 ******/
SET ANSI_NULLS ON
GO
SET QUOTED_IDENTIFIER ON
GO
```


```
/****** Object:  Table [dbo].[tbl_Staff]    Script Date: 14.12.2025. 14:47:29 ******/
SET ANSI_NULLS ON
GO
SET QUOTED_IDENTIFIER ON
GO
```


```
CREATE TABLE [dbo].[tbl_Staff](
    [staff_ID] [int] IDENTITY(1,1) NOT NULL,
    [f_name] [nvarchar](30) NOT NULL,
    [l_name] [nvarchar](30) NOT NULL,
    [position] [nvarchar](30) NOT NULL,
    [salary] [decimal](10, 2) NOT NULL,
    [username] [nvarchar](50) NOT NULL,
    [password] [nvarchar](50) NOT NULL,
    [licenseNo] [nvarchar](20) NULL,
    [weaponPermit] [bit] NULL,
 CONSTRAINT [PK_tbl_Staff] PRIMARY KEY CLUSTERED
(
    [staff_ID] ASC
)WITH (PAD_INDEX = OFF, STATISTICS_NORECOMPUTE = OFF, IGNORE_DUP_KEY = OFF, ALLOW_ROW_LOCKS = ON, ALLOW_PAGE_LOCKS = ON, OPTIMIZE_FOR_SEQUENTIA
 CONSTRAINT [IX_tbl_Staff] UNIQUE NONCLUSTERED
(
    [username] ASC
)WITH (PAD_INDEX = OFF, STATISTICS_NORECOMPUTE = OFF, IGNORE_DUP_KEY = OFF, ALLOW_ROW_LOCKS = ON, ALLOW_PAGE_LOCKS = ON, OPTIMIZE_FOR_SEQUENTIA
) ON [PRIMARY]
GO
```

```
/****** Object:  Table [dbo].[tbl_Ticket]    Script Date: 14.12.2025. 14:47:29 ******/
SET ANSI_NULLS ON
GO
SET QUOTED_IDENTIFIER ON
GO
CREATE TABLE [dbo].[tbl_Ticket](
    [ticket_ID] [int] IDENTITY(1,1) NOT NULL,
    [seatNo] [nvarchar](10) NULL,
    [class] [nvarchar](10) NOT NULL,
    [price] [decimal](5, 2) NOT NULL,
    [booking_ID] [int] NOT NULL,
    [flight_ID] [int] NOT NULL,
 CONSTRAINT [PK_tbl_Ticket] PRIMARY KEY CLUSTERED
(
    [ticket_ID] ASC
)WITH (PAD_INDEX = OFF, STATISTICS_NORECOMPUTE = OFF, IGNORE_DUP_KEY = OFF, ALLOW_ROW_LOCKS = ON, ALLOW_PAGE_LOCKS = ON, OPTIMIZE_FOR_SEQUENTIA
) ON [PRIMARY]
GO
ALTER TABLE [dbo].[tbl_Baggage]  WITH CHECK ADD  CONSTRAINT [FK_tbl_Baggage_tbl_CheckIn] FOREIGN KEY([checkIn_ID])
REFERENCES [dbo].[tbl_CheckIn] ([checkIn_ID])
ON DELETE CASCADE
GO
ALTER TABLE [dbo].[tbl_Baggage] CHECK CONSTRAINT [FK_tbl_Baggage_tbl_CheckIn]
GO
ALTER TABLE [dbo].[tbl_Booking]  WITH CHECK ADD  CONSTRAINT [FK_Booking_Passenger] FOREIGN KEY([passenger_ID])
REFERENCES [dbo].[tbl_Passenger] ([passenger_ID])
GO
ALTER TABLE [dbo].[tbl_Booking] CHECK CONSTRAINT [FK_Booking_Passenger]
GO
ALTER TABLE [dbo].[tbl_CheckIn]  WITH CHECK ADD  CONSTRAINT [FK_tbl_CheckIn_tbl_Ticket] FOREIGN KEY([ticket_ID])
REFERENCES [dbo].[tbl_Ticket] ([ticket_ID])
GO
ALTER TABLE [dbo].[tbl_CheckIn] CHECK CONSTRAINT [FK_tbl_CheckIn_tbl_Ticket]
GO
GO
ALTER TABLE [dbo].[tbl_Flight]  WITH CHECK ADD  CONSTRAINT [FK_tbl_Flight_tbl_Gate] FOREIGN KEY([gate_ID])
REFERENCES [dbo].[tbl_Gate] ([gate_ID])
GO
ALTER TABLE [dbo].[tbl_Flight] CHECK CONSTRAINT [FK_tbl_Flight_tbl_Gate]
GO
ALTER TABLE [dbo].[tbl_Flight]  WITH CHECK ADD  CONSTRAINT [FK_tbl_Flight_tbl_Staff] FOREIGN KEY([airlineStaff_ID])
REFERENCES [dbo].[tbl_Staff] ([staff_ID])
GO
ALTER TABLE [dbo].[tbl_Flight] CHECK CONSTRAINT [FK_tbl_Flight_tbl_Staff]
GO
ALTER TABLE [dbo].[tbl_Payment]  WITH CHECK ADD  CONSTRAINT [FK_tbl_Payment_tbl_Booking] FOREIGN KEY([booking_ID])
REFERENCES [dbo].[tbl_Booking] ([booking_ID])
GO
ALTER TABLE [dbo].[tbl_Payment] CHECK CONSTRAINT [FK_tbl_Payment_tbl_Booking]
GO
ALTER TABLE [dbo].[tbl_SecurityCheck]  WITH CHECK ADD  CONSTRAINT [FK_tbl_SecurityCheck_tbl_CheckIn] FOREIGN KEY([staff_ID])
REFERENCES [dbo].[tbl_Staff] ([staff_ID])
GO
ALTER TABLE [dbo].[tbl_SecurityCheck] CHECK CONSTRAINT [FK_tbl_SecurityCheck_tbl_CheckIn]
GO
ALTER TABLE [dbo].[tbl_Ticket]  WITH CHECK ADD  CONSTRAINT [FK_tbl_Ticket_tbl_Booking] FOREIGN KEY([booking_ID])
REFERENCES [dbo].[tbl_Booking] ([booking_ID])
ON DELETE CASCADE
GO
ALTER TABLE [dbo].[tbl_Ticket] CHECK CONSTRAINT [FK_tbl_Ticket_tbl_Booking]
GO
ALTER TABLE [dbo].[tbl_Ticket]  WITH CHECK ADD  CONSTRAINT [FK_tbl_Ticket_tbl_Flight] FOREIGN KEY([flight_ID])
REFERENCES [dbo].[tbl_Flight] ([flight_ID])
GO
ALTER TABLE [dbo].[tbl_Ticket] CHECK CONSTRAINT [FK_tbl_Ticket_tbl_Flight]
GO
USE [master]
GO
ALTER DATABASE [AirportSystem] SET  READ_WRITE
GO
```

2.4, 2.5, 2.6, 2.7 tbl_Payment, tbl_Security, tbl_Staff, tbl_Ticket

Results Messages

	staff_ID	f_name	l_name	position	salary	username	password	licenseNo	weaponPermit
1	1	Petar	Petrovic	Airline Staff	1500.00	petar_admin	admin123	NULL	NULL
2	2	Miloš	Miloševic	Security Officer	1200.00	milos_sec	sec123	NULL	1
3	3	Jovana	Jovanic	Stewardess	1100.00	jovana_st	stew123	NULL	NULL

	passenger_ID	f_name	l_name	passportNo	UMCN	email	address	contact	username	password
1	1	Mateja	Glišovic	A12345678	0309005782814	mateja.gl12@gmail.com	Marka Markovic 10	0638551101	mateja	sifra123
2	2	Ana	Anic	B98765432	2207999712345	ana.anic@gmail.com	Bulevar Mira 5	0641234567	ana123	pass456
3	3	Nenad	Nenadovic	A32837263	0205993492814	nenad.n@gmail.com	Vojvode Stepe 37	0652536676	Nenad	sif4329

	flight_ID	flightNo	destination	departureTime	arrivalTime	status	gate_ID	airlineStaff_ID
1	1	JU500	New York (JFK)	2025-07-25 06:00:00.000	2025-07-25 18:00:00.000	On Time	1	1
2	2	W6400	London (LTN)	2025-08-01 10:00:00.000	2025-08-01 13:00:00.000	Scheduled	2	1

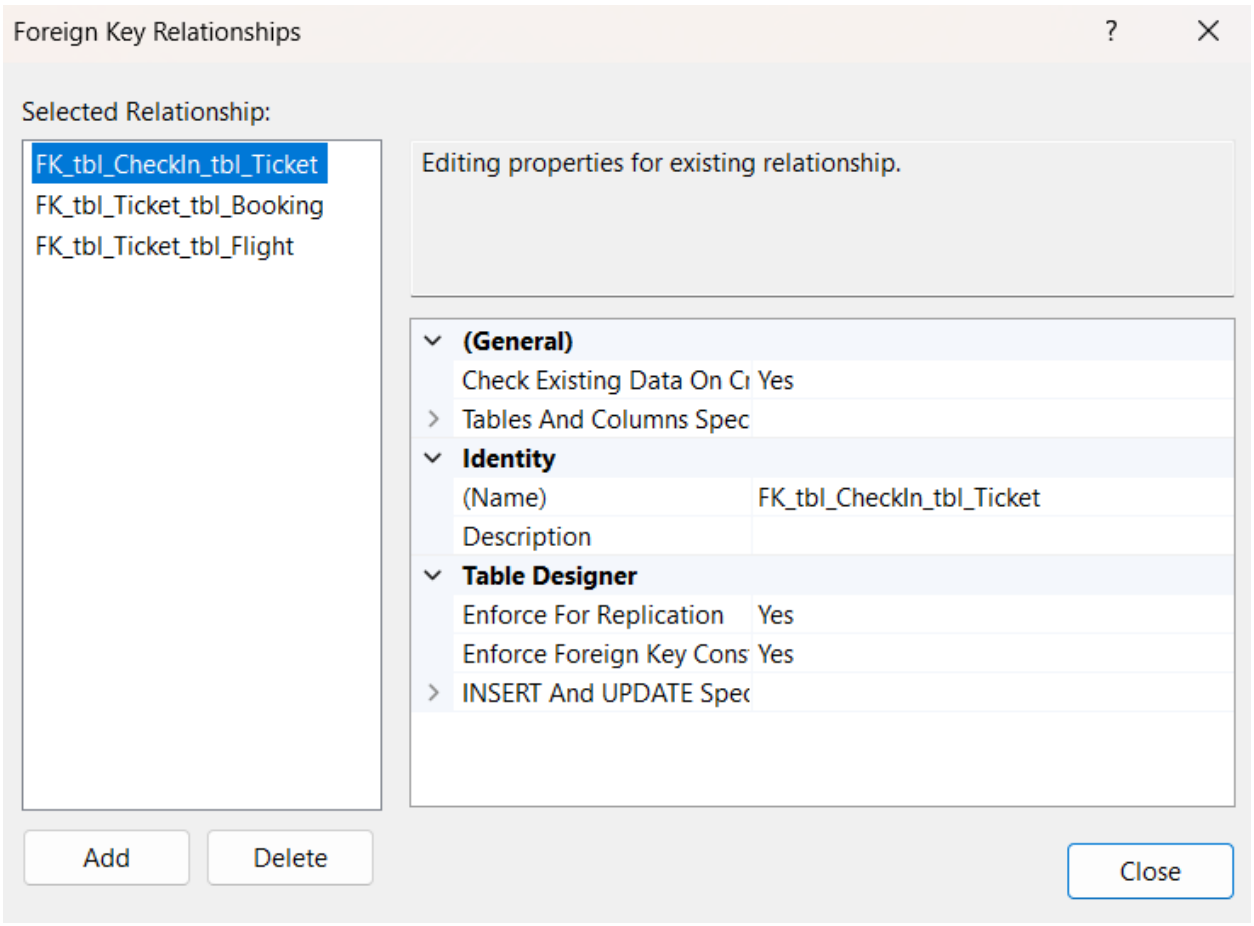
	checkIn_ID	checkinTime	type	boardingPassCode	desk_ID	ticket_ID
1	1	2025-07-25 04:00:00.000	Online	JU500-14F-MATEJA	NULL	1
2	2	2025-08-01 09:00:00.000	Live	W6400-9A-ANA	Counter 12	2

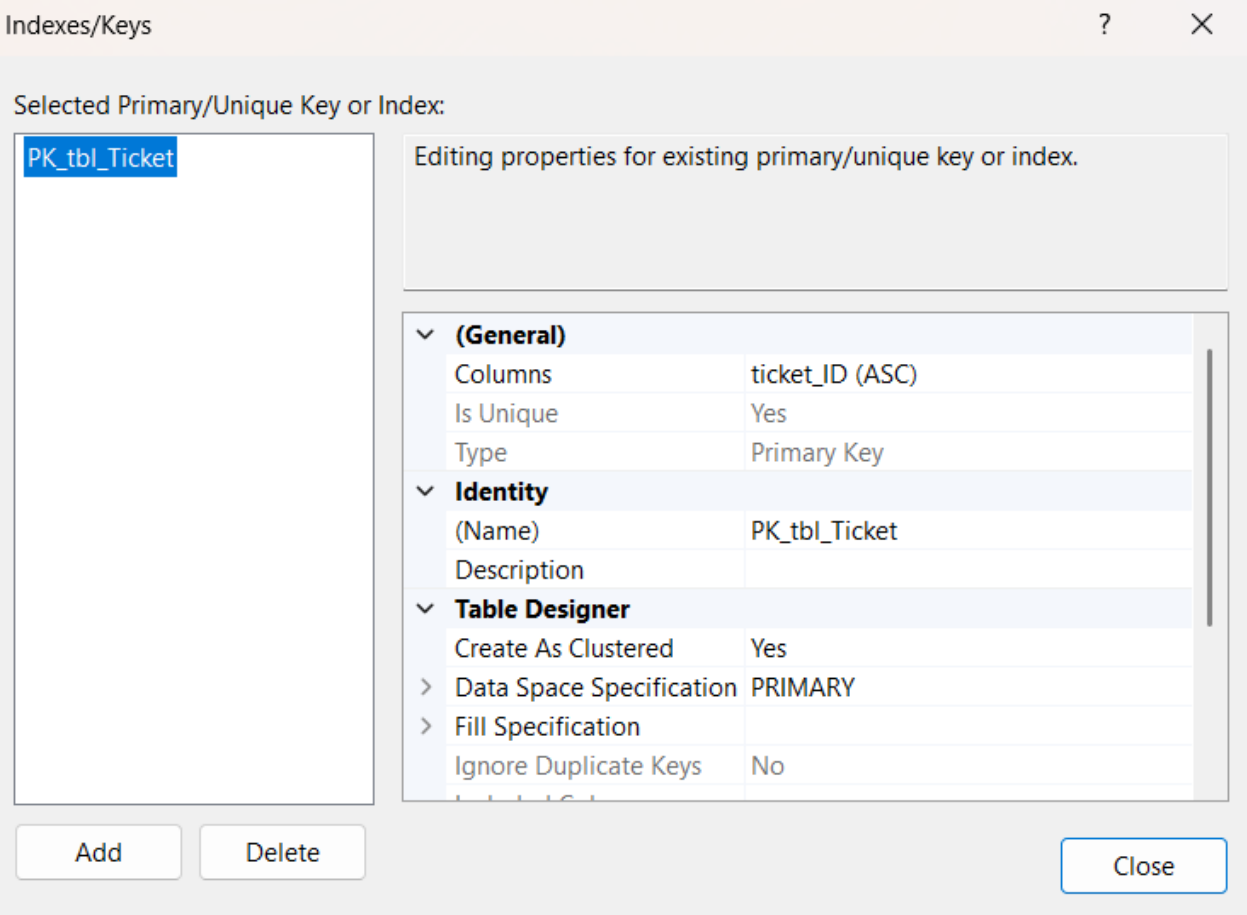
2.8 SQL Table

2.2 Logički model i Normalizacija

Prilikom modeliranja podataka, striktno su poštovana pravila normalizacije kako bi se eliminisala redundansa (ponavljanje) podataka i sprečile anomalije prilikom ažuriranja. Baza je dovedena do **Treće normalne forme (3NF)**:

1. **Prva normalna forma (1NF):** Sve kolone sadrže atomske (nedeljive) vrednosti i svaki zapis je jedinstven.
2. **Druga normalna forma (2NF):** Svi ne-ključni atributi u potpunosti zavise od primarnog ključa.
3. **Treća normalna forma (3NF):** Uklonjene su tranzitivne zavisnosti, tako da ne-ključni atributi zavise isključivo od primarnog ključa, a ne od drugih ne-ključnih atributa.

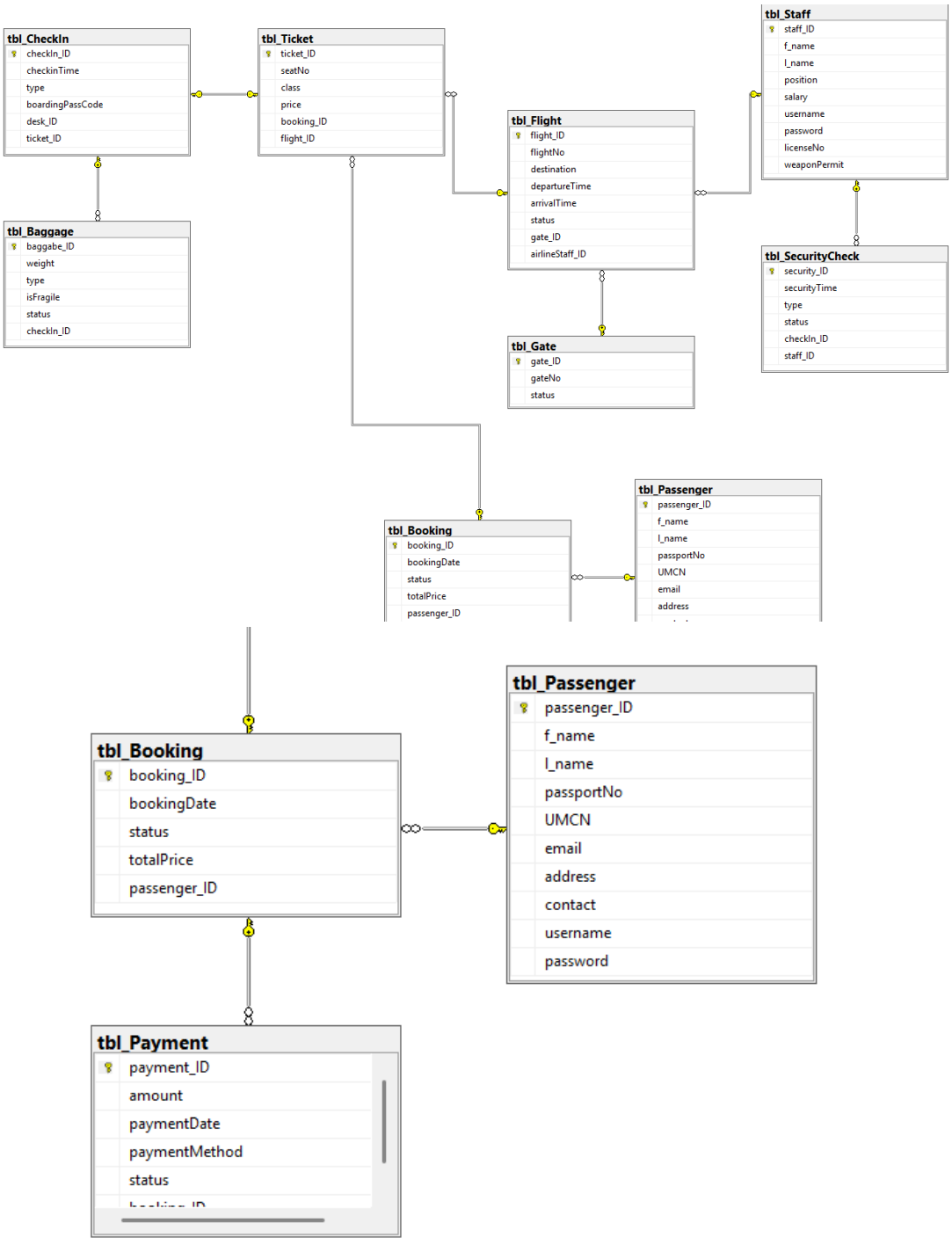

Foreign Key Relationships
Selected Relationship:
FK_tbl_CheckIn_tbl_Ticket
FK_tbl_Ticket_tbl_Booking
FK_tbl_Ticket_tbl_Flight
Editing properties for existing relationship.
(General)
Check Existing Data On Cı Yes
Tables And Columns Spec
Identity
(Name) FK_tbl_CheckIn_tbl_Ticket
Description
Table Designer
Enforce For Replication Yes
Enforce Foreign Key Cons Yes
INSERT And UPDATE Spec
Add
Delete
Close



2.9 Foreign and Index keys

2.3 ER Dijagram (Entity-Relationship Diagram)

Struktura baze podataka i odnosi između entiteta grafički su prikazani na ER dijagramu u nastavku. Dijagram ilustruje veze između ključnih entiteta kao što su Putnici, Letovi, Rezervacije i Karte.



2.10 ER Dijagram

Kao što se vidi na slici 2.10, centralni deo sistema čine tabele `tbl_Booking` i `tbl_Ticket`, koje povezuju putnike (`tbl_Passenger`) sa letovima (`tbl_Flight`). Veze su realizovane

putem stranih ključeva, uz definisana pravila referencijalnog integriteta koja sprečavaju brisanje podataka koji su u upotrebi (npr. nemoguće je obrisati let za koji postoje aktivne karte).

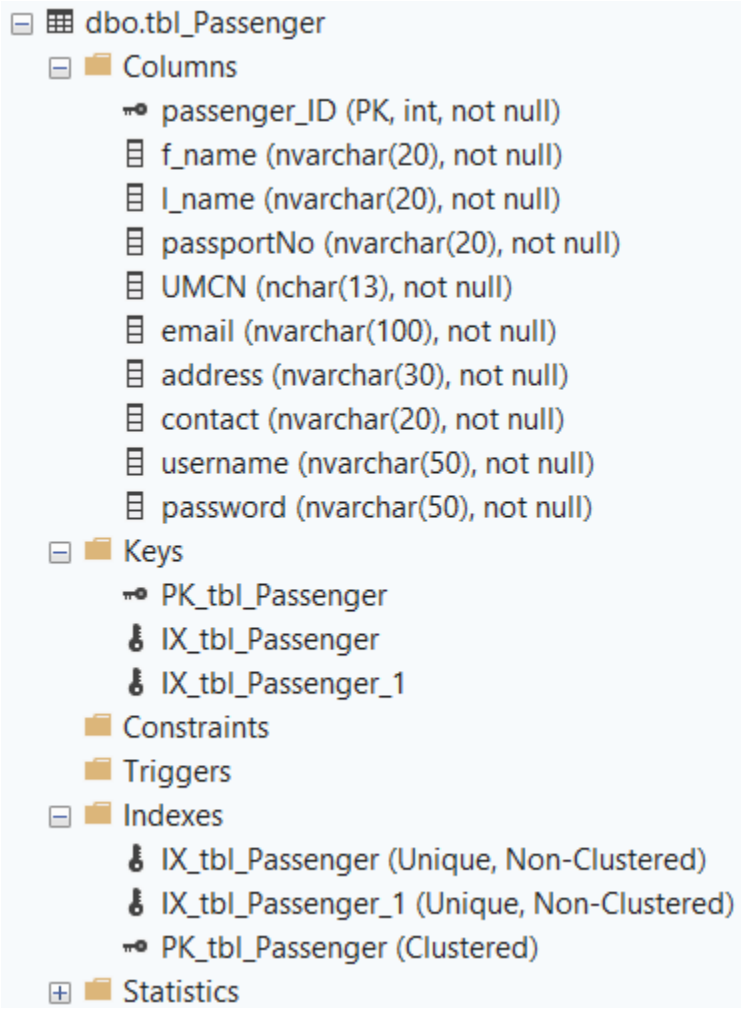
2.4 Rečnik Podataka (Data Dictionary)

U nastavku je dat detaljan opis svih tabela implementiranih u sistemu, sa opisom njihovih atributa i uloga.

2.4.1 Tabela: Putnici (`tbl_Passenger`) Ova tabela predstavlja registar svih korisnika sistema koji koriste usluge prevoza.

- **Namena:** Čuvanje ličnih i pristupnih podataka putnika.
- **Ključni atributi:**
  - `passenger_ID` (INT, PK, Identity): Jedinstveni identifikator putnika.
  - `passportNo` (NVARCHAR): Broj pasoša, služi kao jedinstveni identifikator u stvarnom svetu.

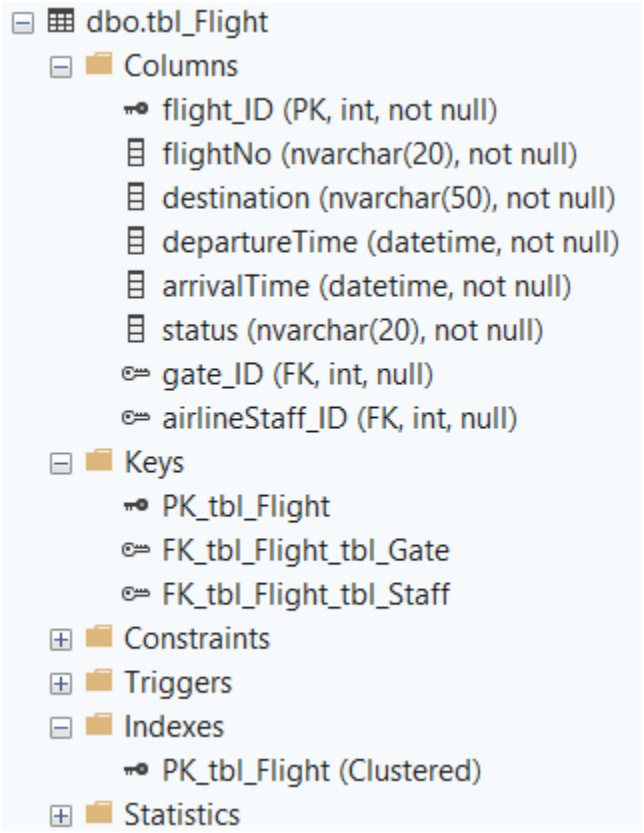
- `username` / `password`: Kredencijali za pristup Putničkom Dashboard-u.



2.11 tbl_Passenger

2.4.2 Tabela: Letovi (`tbl_Flight`) Sadrži red letenja i statusne informacije o letovima.

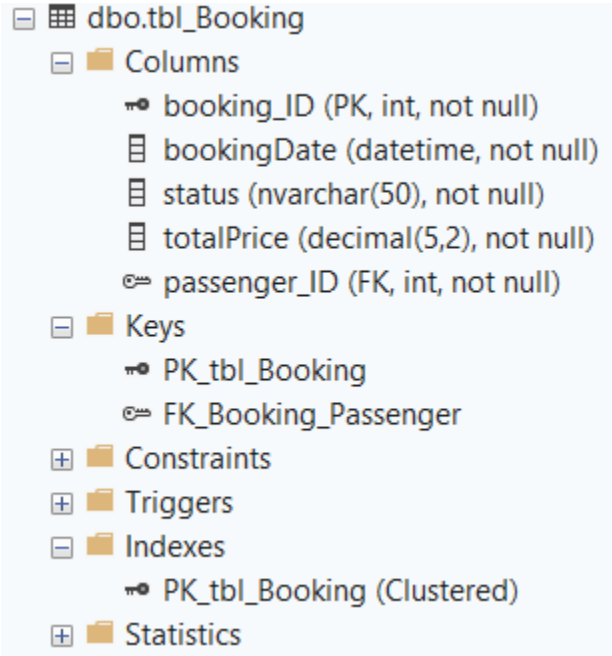
- **Namena:** Definisanje ruta, vremena i statusa letova.
- **Ključni atributi:**
  - `flight_ID` (INT, PK): Interni ID leta.
  - `flightNo` (NVARCHAR): Broj leta (npr. JU500, W6400).
  - `status` (NVARCHAR): Trenutni status leta (On Time, Delayed, Cancelled).
  - `gate_ID` (FK): Veza sa tabelom `tbl_Gate`, definiše na kojoj kapiji se vrši ukrcavanje.



2.12 tbl_Flight

2.4.3 Tabela: Rezervacije (`tbl_Booking`) Ovo je transakciona tabela koja povezuje putnika sa sistemom naplate.

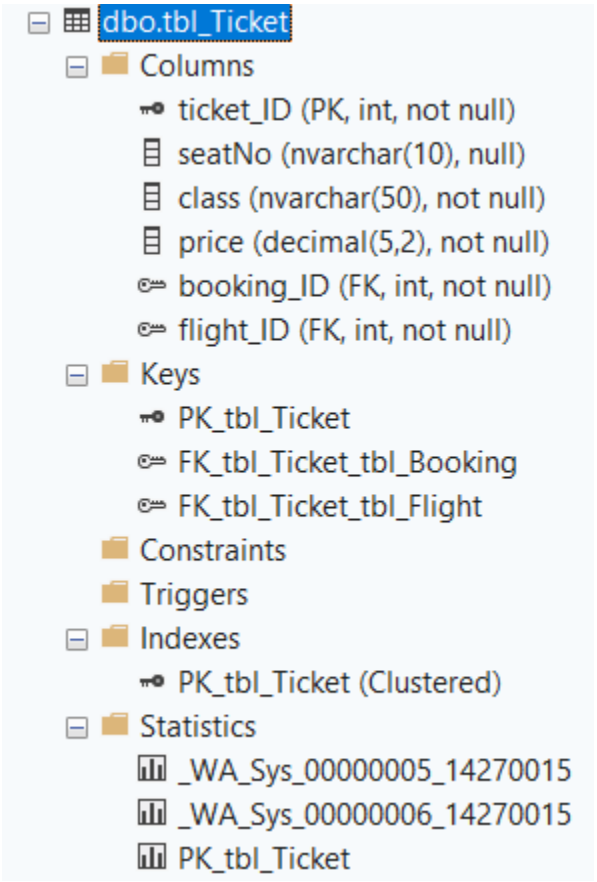
- **Namena:** Praćenje statusa rezervacije i finansijskih zaduženja.
- **Ključni atributi:**
  - `booking_ID` (INT, PK): ID rezervacije.
  - `status` (NVARCHAR): Status procesa (Pending, Confirmed, Canceled).
  - `totalPrice` (DECIMAL): Ukupna cena rezervacije.
  - `passenger_ID` (FK): Referenca na putnika koji je napravio rezervaciju.



2.13 tbl_Booking

2.4.4 Tabela: Karte (`tbl_Ticket`) Predstavlja konkretnu kartu izdatu na osnovu rezervacije.

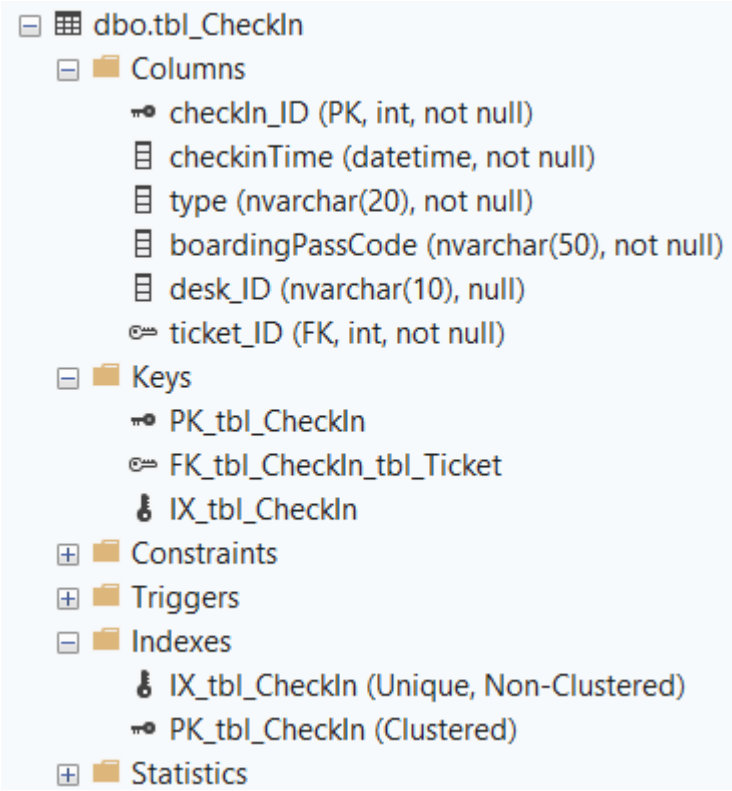
- **Namena:** Dodeljivanje konkretnog sedišta i klase putniku za određeni let.
- **Ključni atributi:**
  - `ticket_ID` (INT, PK): ID karte.
  - `class` (NVARCHAR): Klasa leta (Economy, Business, First). *Napomena: Tokom implementacije, ova kolona je proširena na NVARCHAR(50) kako bi podržala pune nazive klasa.*
  - `seatNo` (NVARCHAR): Oznaka sedišta (npr. 12A).
  - `booking_ID` (FK) i `flight_ID` (FK): Veze ka rezervaciji i letu.



2.14 tbl_ticket

2.4.5 Tabela: Check-In (`tbl_CheckIn`) Sadrži podatke o prijavi putnika na let.

- **Namena:** Generisanje Boarding Pass koda i evidentiranje prisustva putnika.
- **Ključni atributi:**
  - `checkin_ID` (INT, PK).
  - `type` (NVARCHAR): Tip prijave (Live ili Online). Definisano kao `NULLABLE` kako bi se podržale rezervacije koje još nisu procesuirane.
  - `boardingPassCode` (NVARCHAR): Jedinstveni kod generisan u formatu `LET-SEDIŠTE-RANDOM`.



2.15 tbl_CheckIn

3. Implementacija i Logika Aplikacije

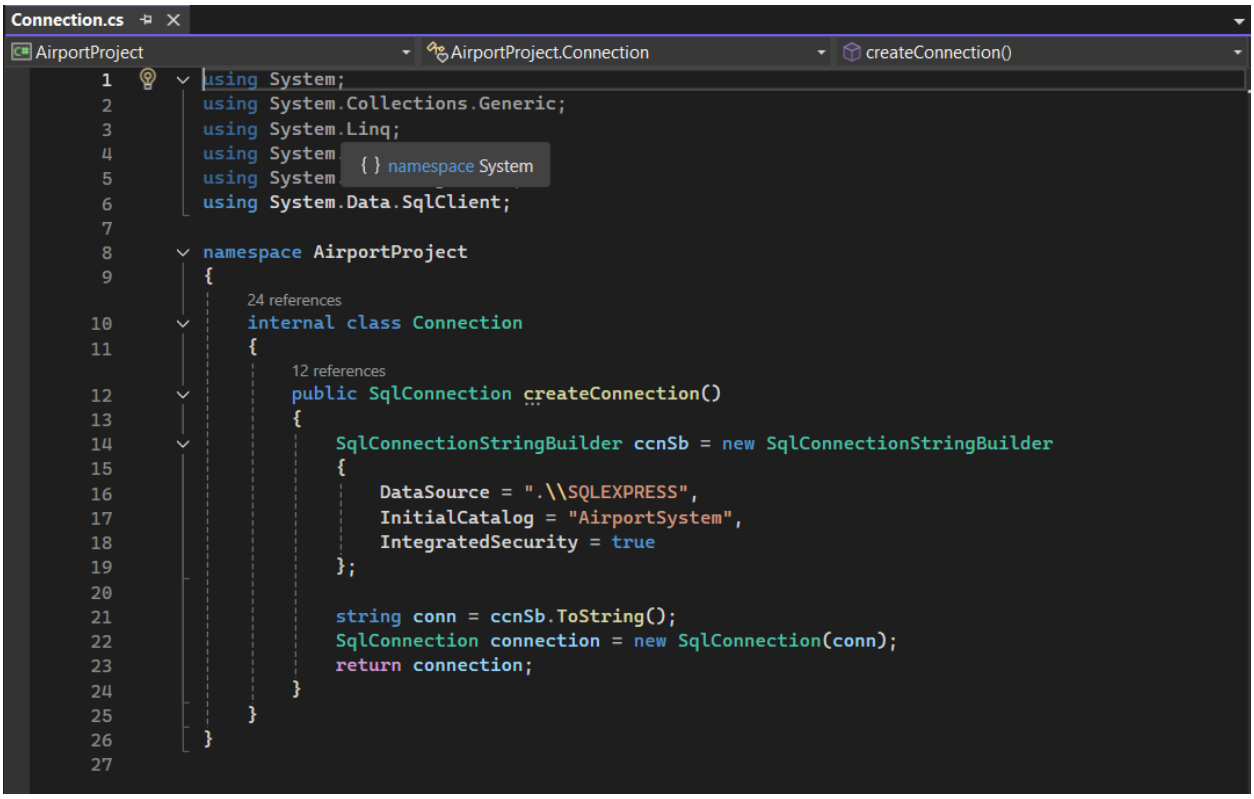
3.1 Tehnološki okvir i Arhitektura

Aplikacija je razvijena koristeći **Microsoft .NET Framework** i programski jezik **C#**. Za korisnički interfejs (UI) korišćena je **WPF (Windows Presentation Foundation)** tehnologija, koja omogućava kreiranje modernog i responzivnog dizajna putem **XAML** (Extensible Application Markup Language) jezika.

Komunikacija sa bazom podataka realizovana je putem **ADO.NET** biblioteke (`System.Data.SqlClient`), koja omogućava direktno izvršavanje SQL upita i manipulaciju podacima. Arhitektura aplikacije je zasnovana na **Event-Driven** (događajima vođenom) modelu, gde se poslovna logika izvršava kao reakcija na akcije korisnika (klik na dugme, izbor iz liste, itd.).

3.2 Klasa za konekciju (`Connection.cs`)

Osnova komunikacije sa bazom podataka je centralizovana klasa `Connection`. Umesto da se konekcioni string ponavlja u svakoj formi, on je definisan na jednom mestu, što olakšava održavanje koda.



3.1 Connection.cs

Ključni mehanizam koji se koristi kroz celu aplikaciju je `using` blok.

- **Objašnjenje:** Korišćenje `using (SqlConnection conn = ...)` osigurava da se konekcija sa bazom automatski zatvori i resursi oslobode čim se blok koda izvrši. Ovo je od vitalnog značaja za sprečavanje curenja memorije ("Memory Leaks") i stabilnost servera.

```
Connection kon = new Connection();

using(SqlConnection conn = kon.createConnection())
{
    conn.Open();
```

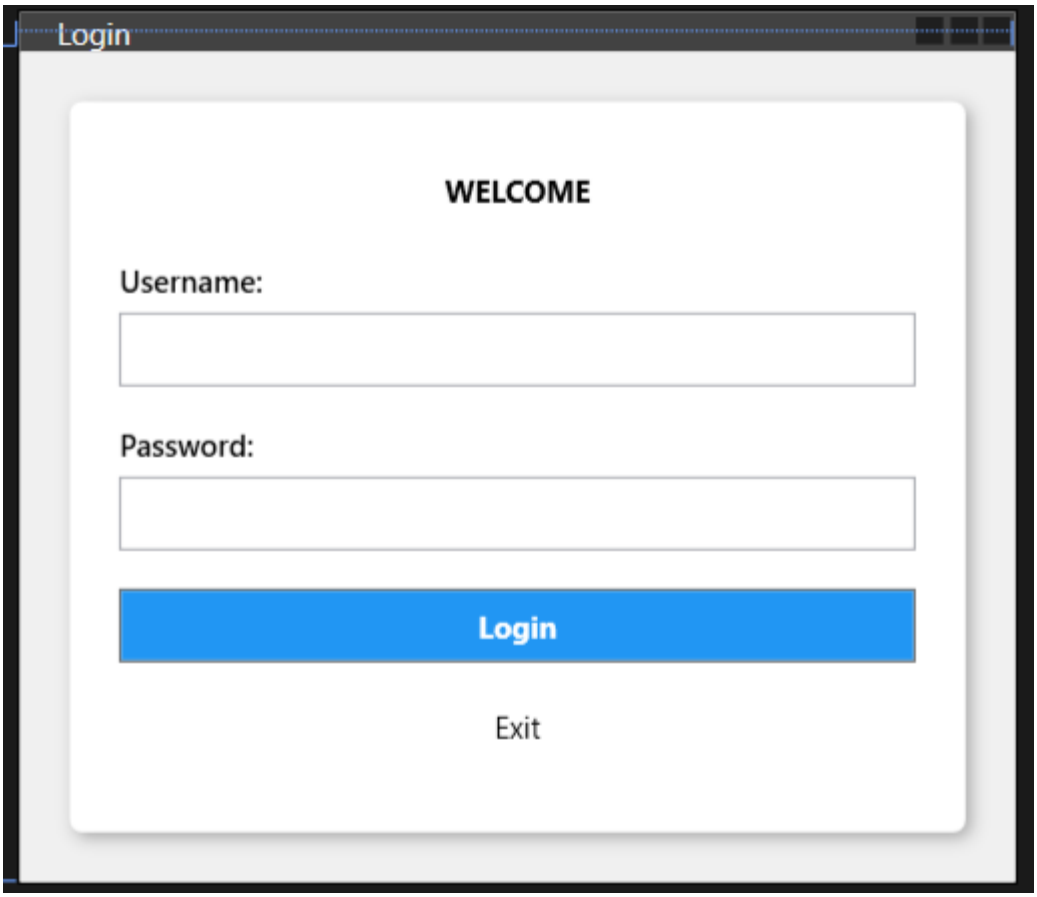
3.2 Using(SqlConnection...)

3.3 Modul za Autentifikaciju (Login Logika)

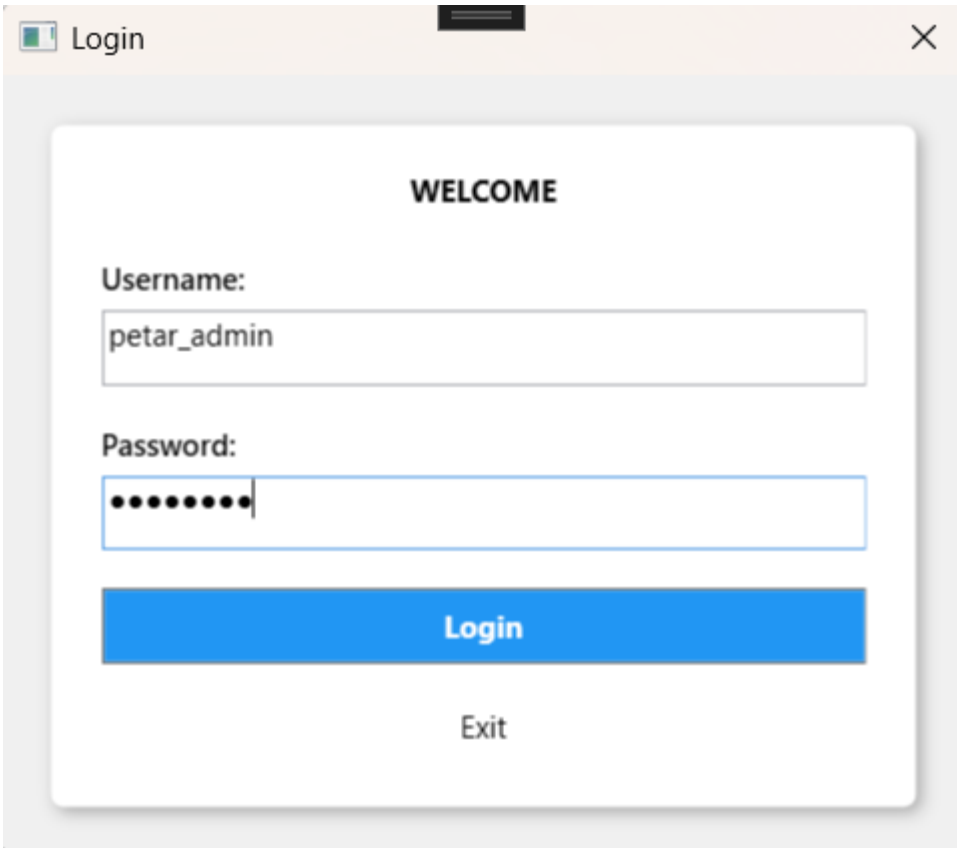
Sigurnost sistema počinje na Login prozoru. Implementirana je dvostepena provera identiteta kako bi se razdvojile uloge Administratora (Osoblja) i Putnika.

Algoritam prijavljivanja:

1. Korisnik unosi korisničko ime i lozinku.
2. Aplikacija prvo šalje upit tabeli `tbl_Staff`.
3. Ukoliko upit vrati rezultat (`ExecuteScalar` != null), otvara se **MainWindow** (Admin Panel).
4. Ukoliko ne vrati rezultat, aplikacija šalje drugi upit tabeli `tbl_Passenger`.
5. Ukoliko tu pronađe poklapanje, otvara se **PassengerDashboard**.
6. U suprotnom, ispisuje se poruka o grešci.



3.3 LoginWindow.xaml



3.4 LoginWindow.xaml (admin)

```
33          private void btnLogin_Click(object sender, RoutedEventArgs e)
34          {
35              string userinput = txtUsername.Text;
36              string passinput = txtPassword.Password;
37
38              if(string.IsNullOrEmpty(userinput) || string.IsNullOrEmpty(passinput))
39              {
40                  MessageBox.Show("Please enter both username and password.");
41                  return;
42              }
43
44              try
45              {
46                  Connection kon = new Connection();
47
48                  using(SqlConnection conn = kon.createConnection())
49                  {
50                      conn.Open();
51
52                      // checking if it's staff (admin)
53                      string queryStaff = "SELECT staff_ID FROM tbl_staff WHERE username=@user AND password=@pass";
54                      SqlCommand cmdStaff = new SqlCommand(queryStaff, conn);
55                      cmdStaff.Parameters.AddWithValue("@user", userinput);
56                      cmdStaff.Parameters.AddWithValue("@pass", passinput);
57
58                      object resultStaff = cmdStaff.ExecuteScalar();
59
60                      // if we found someone
61                      if(resultStaff != null)
62                      {
63                          MessageBox.Show($"Welcome {userinput}");
64                          MainWindow adminPanel = new MainWindow();
65                          adminPanel.Show();
66                          this.Close();
67                          return;
68                      }
```

```
70                      // checking if it's a passenger
71
72                      string queryPass = "SELECT passenger_ID FROM tbl_passenger WHERE username=@user AND password=@pass";
73                      SqlCommand cmdPass = new SqlCommand(queryPass, conn);
74
75                      cmdPass.Parameters.AddWithValue("@user", userinput);
76                      cmdPass.Parameters.AddWithValue("@pass", passinput);
77
78                      object resultPass = cmdPass.ExecuteScalar();
79
80                      if(resultPass != null)
81                      {
82                          int foundID = Convert.ToInt32(resultPass);
83                          MessageBox.Show($"Welcome {userinput}!");
84
85                          PassengerDashboard userPanel = new PassengerDashboard(foundID, userinput);
86                          userPanel.Show();
87                          this.Close();
88                          return;
89                      }
90                      else
91                      {
92                          MessageBox.Show("Invalid username or password.", "Login Failed", MessageBoxButton.OK, MessageBoxImage.
93                      }
94                  }
95              }
96              catch (Exception ex)
97              {
98                  MessageBox.Show("Error: " +  ex.Message);
99              }
100         }
101     }
102 }
103
```

3.5 btnLogin_Click

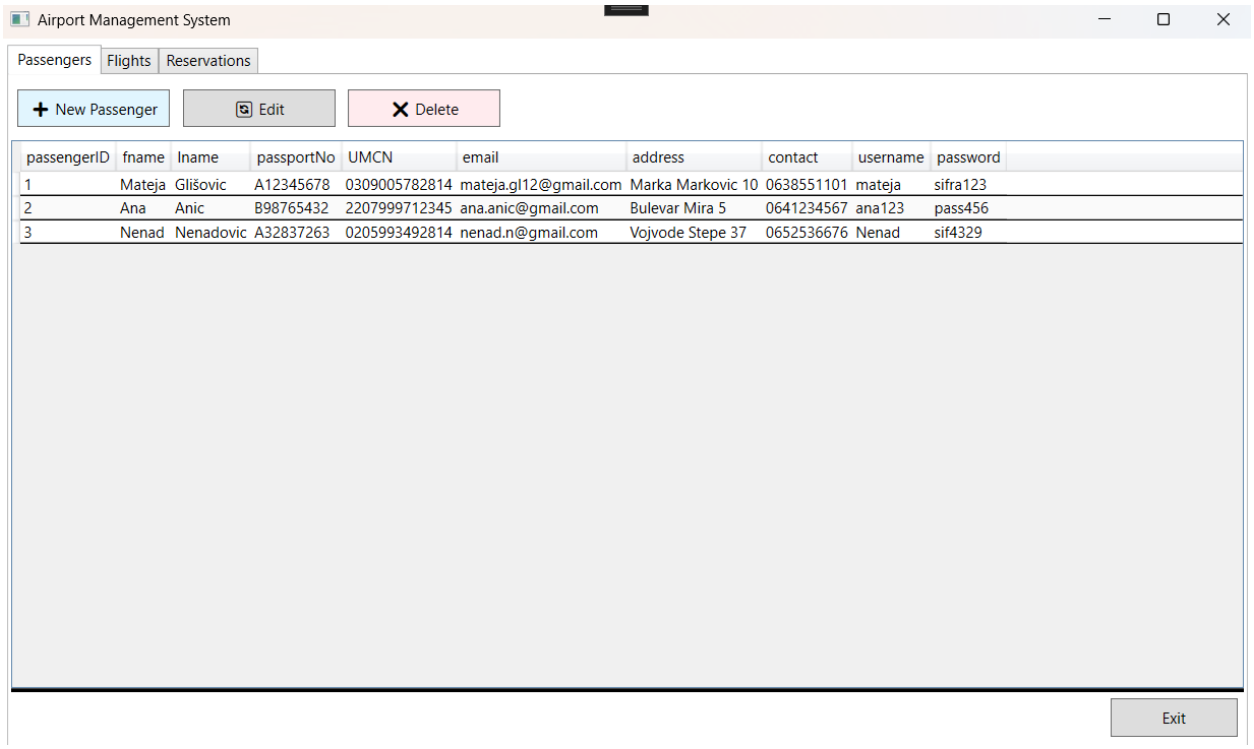
3.4 Centralni Administrativni Panel (`MainWindow`)

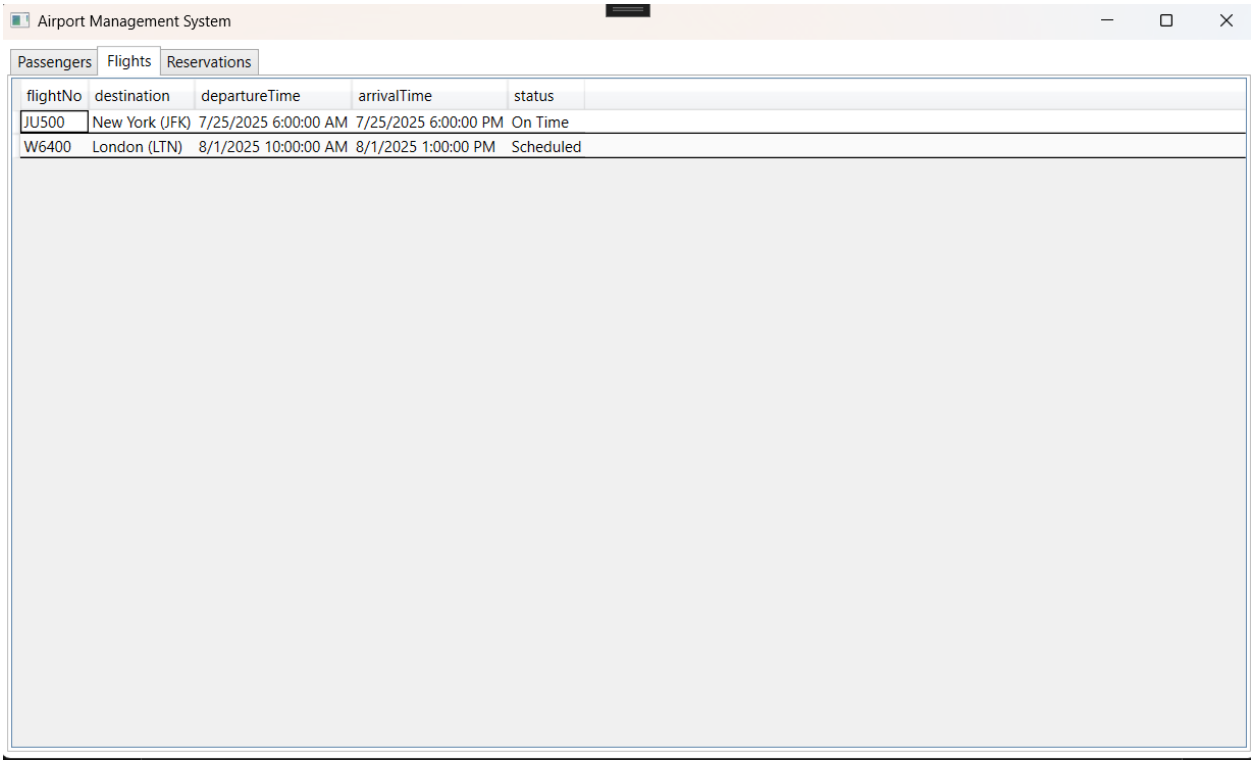
Glavni prozor aplikacije (`MainWindow`) dizajniran je kao centralizovana konzola za upravljanje sistemom. Umesto otvaranja desetina malih prozora, implementiran je **`TabControl`** mehanizam koji omogućava administratoru da se brzo prebacuje između tri ključna entiteta:

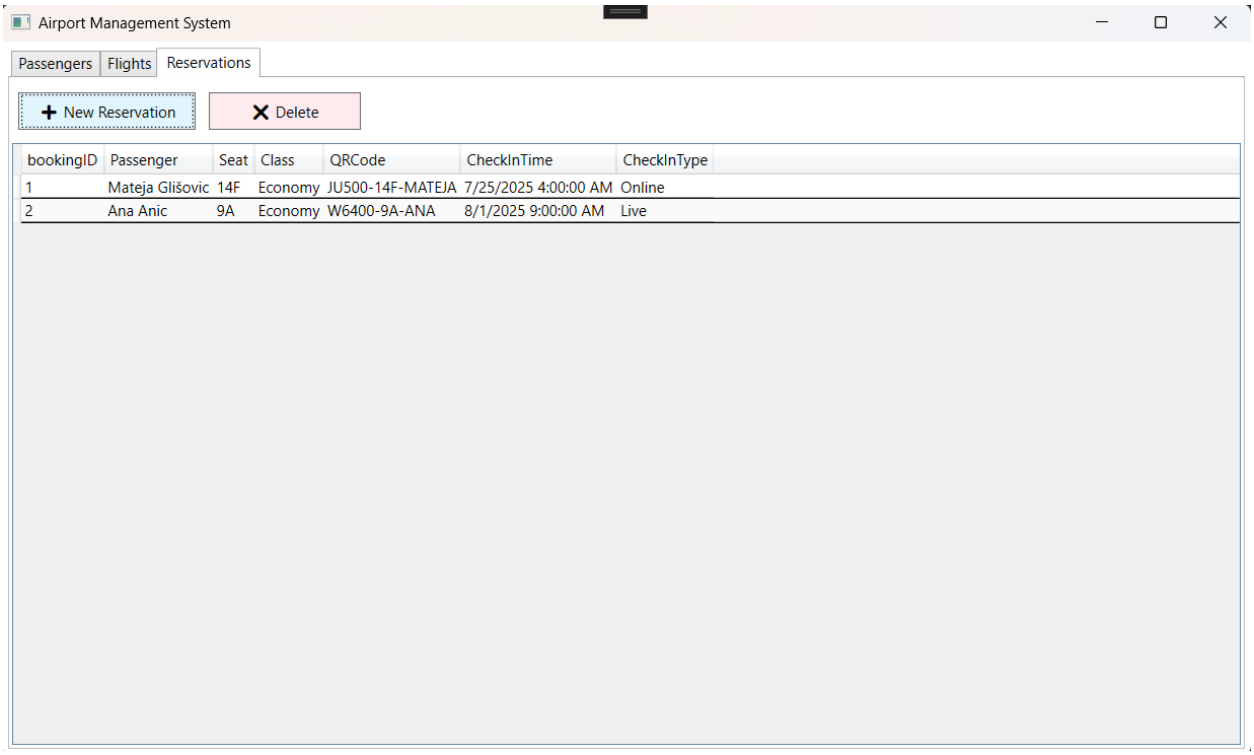
1. **Putnici (Passengers)**
2. **Letovi (Flights)**
3. **Rezervacije (Reservations)**

Tehnička implementacija prikaza podataka: Za prikaz podataka korišćena je kontrola **`DataGrid`**. Podaci se ne unose ručno u tabelu, već se dinamički učitavaju iz baze podataka korišćenjem **`SqlDataAdapter`** klase.

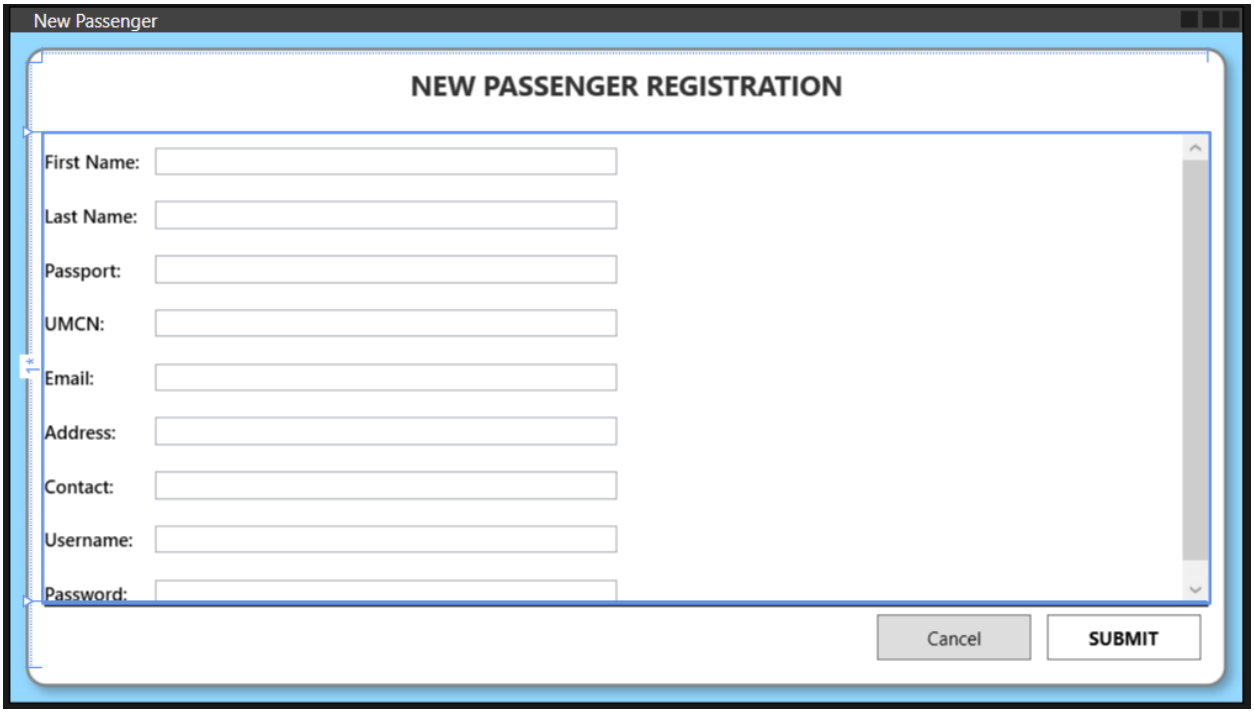
- **Logika učitavanja (`LoadPassengers`):** Metoda funkcioniše po principu "Disconnected Architecture". `SqlDataAdapter` preuzima podatke iz baze, puni lokalnu tabelu u memoriji (`DataTable`), a zatim se ta tabela "vezuje" (Binding) za `DataGrid` na ekranu. Ovo omogućava brz rad aplikacije jer ne držimo otvorenu konekciju sa bazom dok korisnik samo gleda podatke.


Airport Management System
Passengers
Flights
Reservations
New Passenger
Edit
Delete
passengerID fname lname passportNo UMCN email address contact username password
1 Mateja Glišovic A12345678 0309005782814 mateja.gl12@gmail.com Marka Markovic 10 0638551101 mateja sifra123
2 Ana Anic B98765432 2207999712345 ana.anic@gmail.com Bulevar Mira 5 0641234567 ana123 pass456
3 Nenad Nenadovic A32837263 0205993492814 nenad.n@gmail.com Vojvode Stepe 37 0652536676 Nenad sif4329
Exit

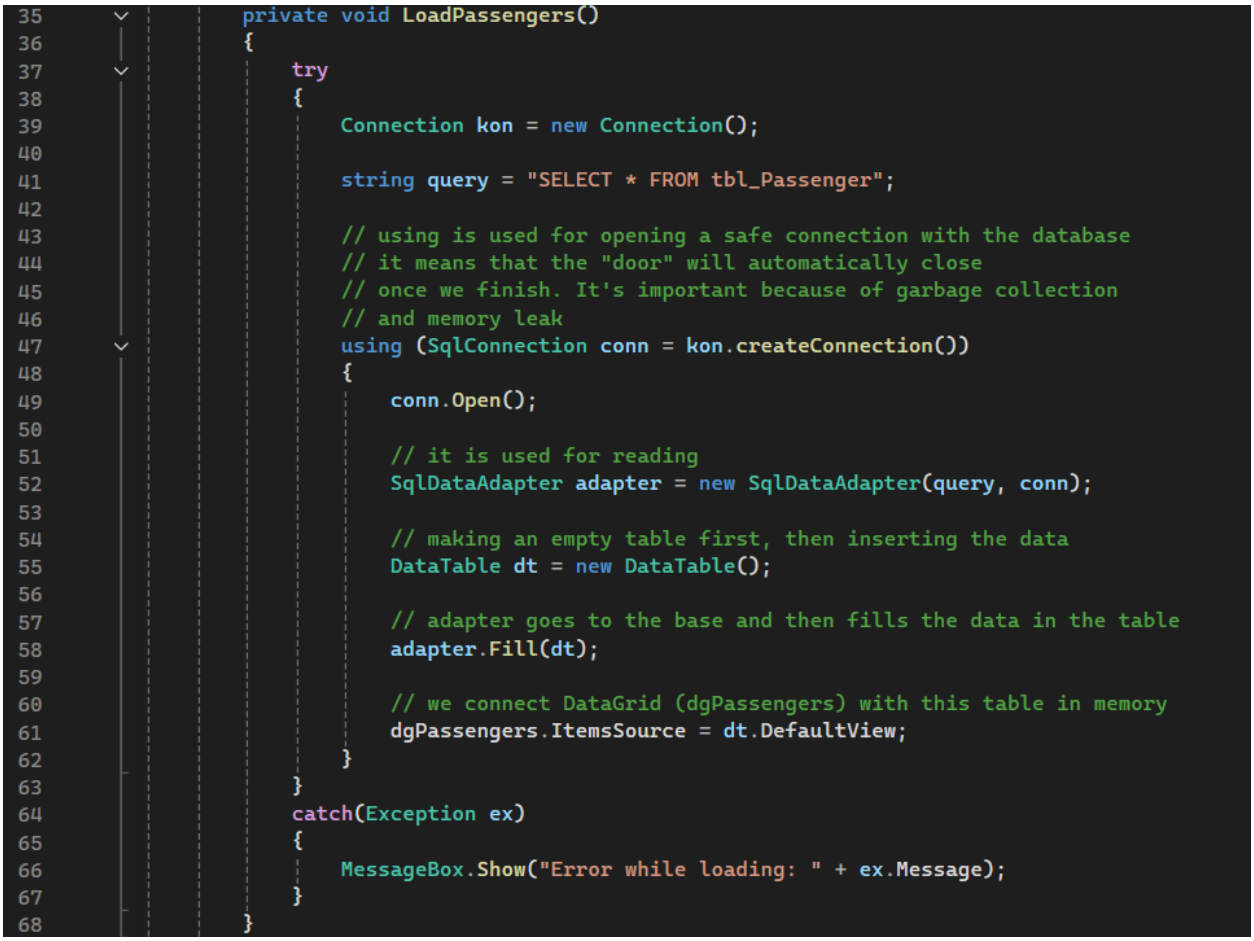

Airport Management System
Passengers
Flights
Reservations
flightNo destination departureTime arrivalTime status
JU500 New York (JFK) 7/25/2025 6:00:00 AM 7/25/2025 6:00:00 PM On Time
W6400 London (LTN) 8/1/2025 10:00:00 AM 8/1/2025 1:00:00 PM Scheduled



3.6 MainWindow.xaml



3.7 AddPassengerWindow.xaml



```
35      private void LoadPassengers()
36      {
37          try
38          {
39              Connection kon = new Connection();
40
41              string query = "SELECT * FROM tbl_Passenger";
42
43              // using is used for opening a safe connection with the database
44              // it means that the "door" will automatically close
45              // once we finish. It's important because of garbage collection
46              // and memory leak
47              using (SqlConnection conn = kon.createConnection())
48              {
49                  conn.Open();
50
51                  // it is used for reading
52                  SqlDataAdapter adapter = new SqlDataAdapter(query, conn);
53
54                  // making an empty table first, then inserting the data
55                  DataTable dt = new DataTable();
56
57                  // adapter goes to the base and then fills the data in the table
58                  adapter.Fill(dt);
59
60                  // we connect DataGrid (dgPassengers) with this table in memory
61                  dgPassengers.ItemsSource = dt.DefaultView;
62              }
63          }
64          catch(Exception ex)
65          {
66              MessageBox.Show("Error while loading: " + ex.Message);
67          }
68      }
```

3.8 LoadPassenger

3.5 Upravljanje Putnicima (CRUD Operacije)

Najvažniji administrativni proces je upravljanje bazom putnika. Implementiran je kompletan **CRUD** (Create, Read, Update, Delete) sistem.

3.6.1 Kreiranje novog putnika (`PassengerWindow`)

Klikom na dugme "Add Passenger", otvara se modalni prozor `PassengerWindow`. Ovaj prozor služi za unos ličnih podataka (Ime, Prezime, Broj pasoša, Kontakt).

Ključni aspekti implementacije:

1. **Validacija unosa:** Pre nego što se podaci pošalju bazi, C# kod proverava da li su sva polja popunjena. Ako nisu, izvršenje se prekida (`return`) i korisnik dobija obaveštenje.
2. **SQL INSERT komanda:** Podaci se šalju bazi putem parametrizovanog `INSERT INTO` upita.
3. **Automatski ID:** Baza podataka je konfigurisana da sama generiše `passenger_ID` (Identity Specification), čime se garantuje jedinstvenost svakog putnika.

```
private void btnSubmit_Click(object sender, RoutedEventArgs e)
{
    if(f_name.Text == "" || l_name.Text == ""
        || passportNo.Text == "" || UMCN.Text == ""
        || email.Text == "" || address.Text == ""
        || contact.Text == "" || username.Text  == ""
        || password.Password == "")
    {
        MessageBox.Show("Please fill all the required fields");
        return;
    }

    try
    {
        Connection kon = new Connection();
        string query = "";

        if (passenger_ID == 0)
        {
            query = @"INSERT INTO tbl_passenger
                        (f_name, l_name, passportNo, umcn, email, address, contact, username, password)
                        VALUES
                        (@f_name, @l_name, @passportNo, @umcn, @email, @address, @contact, @username, @password)";
        }
        else
        {
            query = @"UPDATE tbl_passenger
                        SET f_name=@f_name, l_name=@l_name, passportNo=@passportNo,
                        umcn=@umcn, email=@email, address=@address, contact=@contact,
                        username=@username, password=@password
                    WHERE passenger_ID=@id";

        }
        using (SqlConnection conn = kon.createConnection())
        {
            // Opening the database
            conn.Open();
            SqlCommand cmd = new SqlCommand(query, conn);

            // Connecting textbox with the database
            cmd.Parameters.AddWithValue("@f_name", f_name.Text);
            cmd.Parameters.AddWithValue("@l_name", l_name.Text);
            cmd.Parameters.AddWithValue("@passportNo", passportNo.Text);
            cmd.Parameters.AddWithValue("@umcn", UMCN.Text);
            cmd.Parameters.AddWithValue("@email", email.Text);
            cmd.Parameters.AddWithValue("@address", address.Text);
            cmd.Parameters.AddWithValue("@contact", contact.Text);
            cmd.Parameters.AddWithValue("@username", username.Text);

            cmd.Parameters.AddWithValue("@password", password.Password);

            // if update, then add ID parameter
            if(passenger_ID != 0)
            {
                cmd.Parameters.AddWithValue("@id", passenger_ID);
            }

            // EXECUTE (insert to the database)
            cmd.ExecuteNonQuery();

            MessageBox.Show("Passenger successfully added!", "Success", MessageBoxButton.OK, MessageBoxImage.Information);
            this.Close();
        }
    }
    catch (Exception ex)
    {
        MessageBox.Show("Error saving data: " + ex.Message, "Database Error", MessageBoxButton.OK, MessageBoxImage.Error);
    }
```

3.9 btnSubmit_Click

3.6.2 Ponovno korišćenje koda (Edit vs Create)

Jedna od naprednih karakteristika sistema je **reupotreba iste forme** za kreiranje i izmenu putnika.

- Ukoliko se konstruktor pozove bez parametara (`new PassengerWindow()`), forma je u modu za **kreiranje**.
- Ukoliko se konstruktor pozove sa podacima (`new PassengerWindow(DataRowView row)`), forma se automatski popunjava podacima selektovanog putnika i prebacuje se u mod za **ažuriranje (Update)**.

Ovim pristupom smanjen je broj linija koda i olakšano održavanje aplikacije.

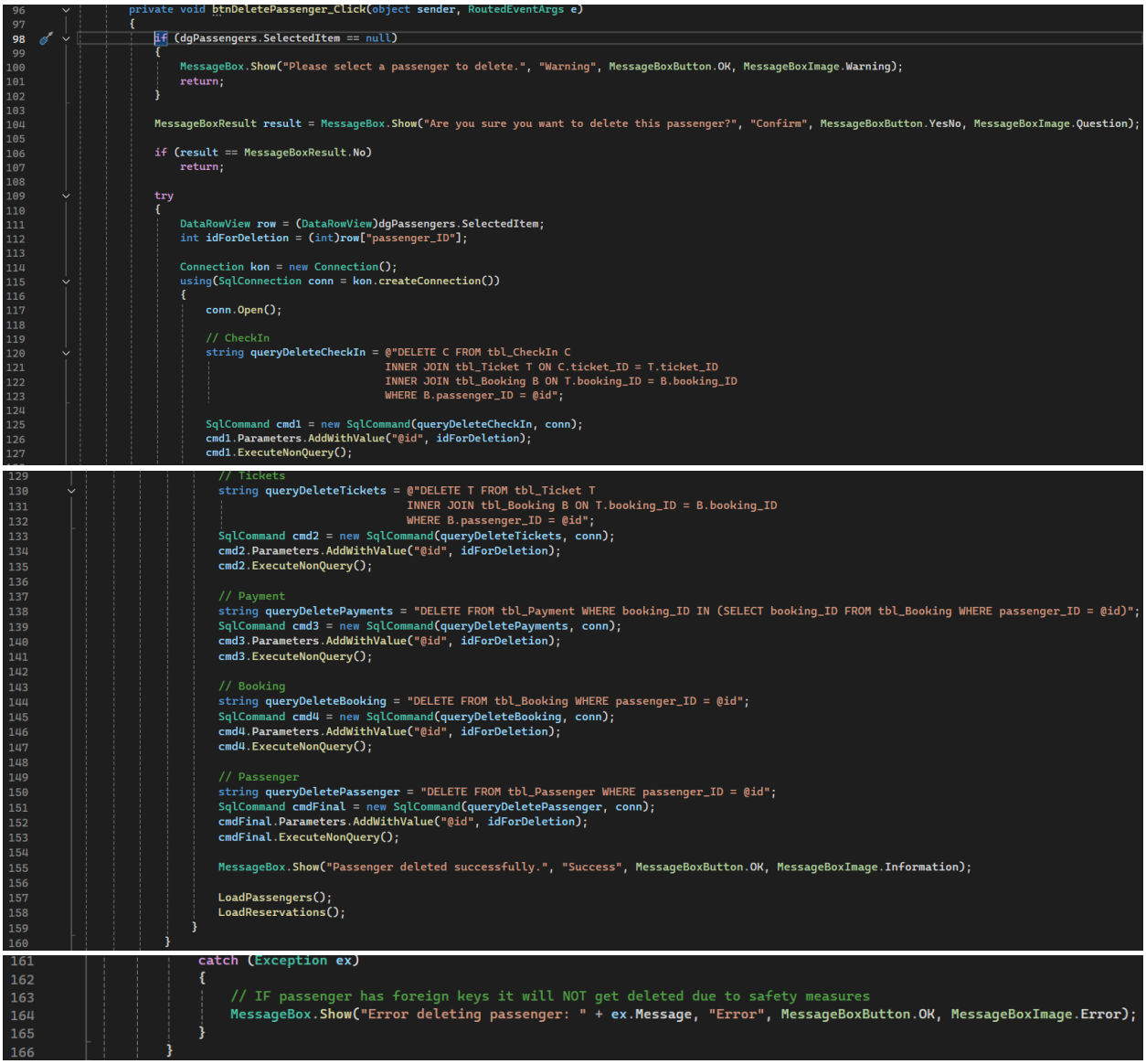
3.6.3 Kaskadno brisanje putnika (Safety First)

Brisanje putnika iz baze nije trivijalna operacija zbog relacija sa drugim tabelama (Strani ključevi). Ako bismo obrisali putnika koji ima rezervacije, baza bi prijavila grešku integriteta.

Zbog toga je implementiran **algoritam sigurnog brisanja**:

1. Prvo se brišu svi **Check-In** zapisi vezani za tog putnika.
2. Zatim se brišu sve **Karte (Tickets)**.
3. Zatim se brišu **Rezervacije (Bookings)** i uplate.
4. Tek kada putnik više nema vezanih podataka, briše se sam zapis iz tabele `tbl_Passenger`.

Ovaj mehanizam sprečava pojavu "siročića" (Orphan records) u bazi podataka i osigurava stabilnost sistema.



```
private void btnDeletePassenger_Click(object sender, RoutedEventArgs e)
{
    if (dgPassengers.SelectedItem == null)
    {
        MessageBox.Show("Please select a passenger to delete.", "Warning", MessageBoxButton.OK, MessageBoxImage.Warning);
        return;
    }

    MessageBoxResult result = MessageBox.Show("Are you sure you want to delete this passenger?", "Confirm", MessageBoxButton.YesNo, MessageBoxImage.Question);

    if (result == MessageBoxResult.No)
        return;

    try
    {
        DataRowView row = (DataRowView)dgPassengers.SelectedItem;
        int idForDeletion = (int)row["passenger_ID"];

        Connection kon = new Connection();
        using(SqlConnection conn = kon.createConnection())
        {
            conn.Open();

            // CheckIn
            string queryDeleteCheckIn = @"DELETE C FROM tbl_CheckIn C
                                          INNER JOIN tbl_Ticket T ON C.ticket_ID = T.ticket_ID
                                          INNER JOIN tbl_Booking B ON T.booking_ID = B.booking_ID
                                          WHERE B.passenger_ID = @id";

            SqlCommand cmd1 = new SqlCommand(queryDeleteCheckIn, conn);
            cmd1.Parameters.AddWithValue("@id", idForDeletion);
            cmd1.ExecuteNonQuery();

            // Tickets
            string queryDeleteTickets = @"DELETE T FROM tbl_Ticket T
                                          INNER JOIN tbl_Booking B ON T.booking_ID = B.booking_ID
                                          WHERE B.passenger_ID = @id";
            SqlCommand cmd2 = new SqlCommand(queryDeleteTickets, conn);
            cmd2.Parameters.AddWithValue("@id", idForDeletion);
            cmd2.ExecuteNonQuery();

            // Payment
            string queryDeletePayments = "DELETE FROM tbl_Payment WHERE booking_ID IN (SELECT booking_ID FROM tbl_Booking WHERE passenger_ID = @id)";
            SqlCommand cmd3 = new SqlCommand(queryDeletePayments, conn);
            cmd3.Parameters.AddWithValue("@id", idForDeletion);
            cmd3.ExecuteNonQuery();

            // Booking
            string queryDeleteBooking = "DELETE FROM tbl_Booking WHERE passenger_ID = @id";
            SqlCommand cmd4 = new SqlCommand(queryDeleteBooking, conn);
            cmd4.Parameters.AddWithValue("@id", idForDeletion);
            cmd4.ExecuteNonQuery();

            // Passenger
            string queryDeletePassenger = "DELETE FROM tbl_Passenger WHERE passenger_ID = @id";
            SqlCommand cmdFinal = new SqlCommand(queryDeletePassenger, conn);
            cmdFinal.Parameters.AddWithValue("@id", idForDeletion);
            cmdFinal.ExecuteNonQuery();

            MessageBox.Show("Passenger deleted successfully.", "Success", MessageBoxButton.OK, MessageBoxImage.Information);

            LoadPassengers();
            LoadReservations();
        }
    }
    catch (Exception ex)
    {
        // IF passenger has foreign keys it will NOT get deleted due to safety measures
        MessageBox.Show("Error deleting passenger: " + ex.Message, "Error", MessageBoxButton.OK, MessageBoxImage.Error);
    }
}
```

3.10 btnDeletePassenger_Click

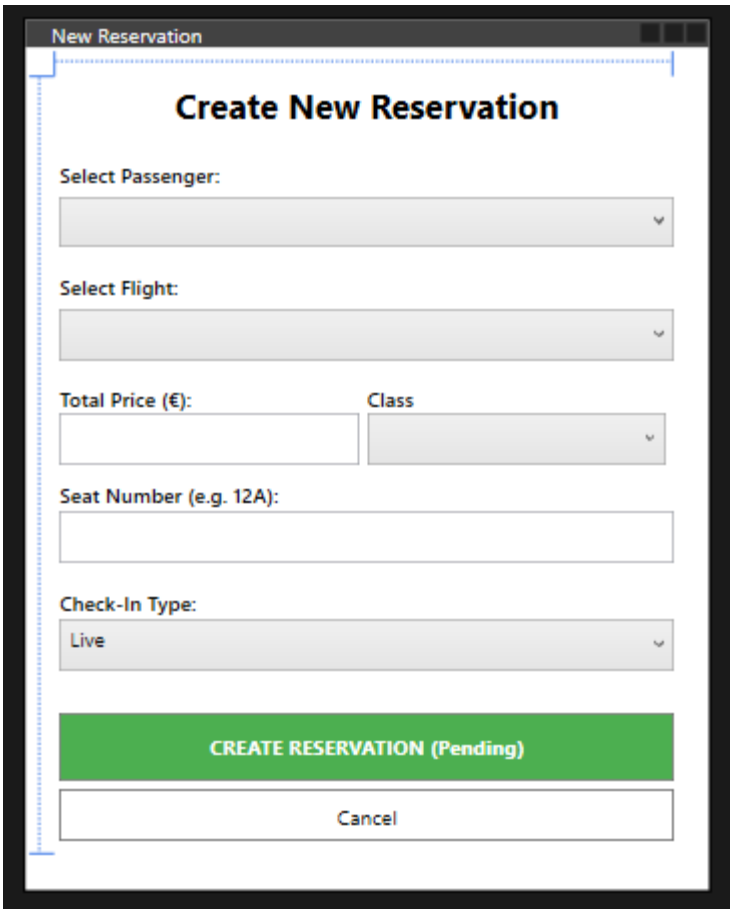
3.7 Admin Panel: Transakciona obrada podataka

Najkompleksniji deo sistema je kreiranje nove rezervacije. Ovaj proces zahteva očuvanje integriteta podataka kroz više tabela istovremeno.

Proces kreiranja rezervacije (`AddReservationWindow`): Da bi se rezervacija uspešno kreirala, sistem izvršava seriju zavisnih koraka unutar jedne konekcije:

1. **Validacija:** Proverava se da li su sva polja (Putnik, Let, Klasa, Sedište) popunjena.
2. **Provera duplikata:** Pre upisa, sistem izvršava `COUNT(*)` upit kojim proverava da li isti putnik već ima rezervaciju za isti let. Ovo sprečava logičke greške u poslovanju.
3. **Upis Rezervacije (Booking):** Prvo se upisuje podatak u `tbl_Booking`. Koristeći komandu `SELECT SCOPE_IDENTITY()`, sistem odmah preuzima ID novokreirane rezervacije.
4. **Upis Karte (Ticket):** Preuzeti `booking_ID` se koristi za kreiranje karte, čime se ostvaruje relacija 1:N.
5. **Upis Prijave (Check-In):** Na kraju, generiše se zapis u `tbl_CheckIn` sa statusom "Online" ili "Live".

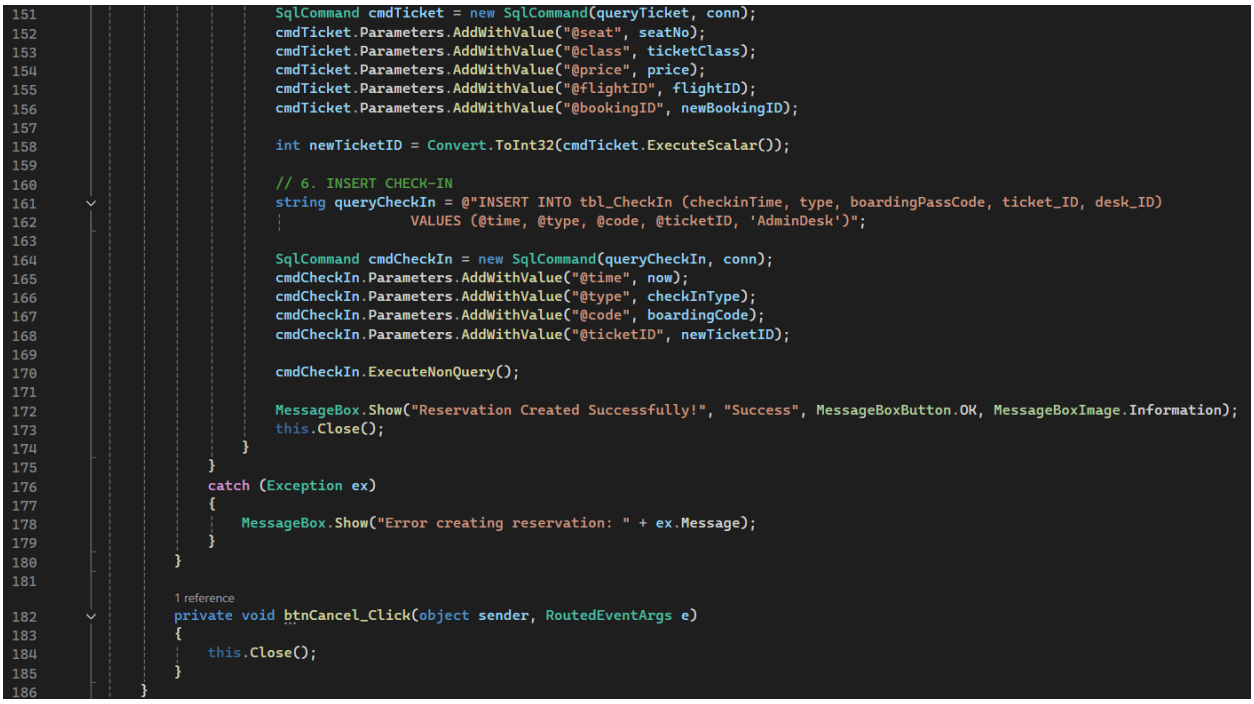
Ovaj pristup ("Chain Reaction") osigurava da nikada nećemo imati kartu bez rezervacije, niti check-in zapis bez karte.



3.11 AddReservationWindow.xaml

```
79      private void btnSave_Click(object sender, RoutedEventArgs e)
80      {
81          if (cbPassengers.SelectedValue == null ||
82          cbFlights.SelectedValue == null ||
83          cbClass.SelectedItem == null ||
84          cbCheckInType.SelectedItem == null ||
85          string.IsNullOrEmpty(txtPrice.Text) ||
86          string.IsNullOrEmpty(txtSeat.Text))
87          {
88              MessageBox.Show("Please fill ALL fields (Passenger, Flight, Price, Seat, Class, Check-In Type).", "Error", MessageBoxButton.OK, MessageBoxImage.Error)
89              return;
90          }
91
92          try
93          {
94              // getting the passengers
95              int passengerID = (int)cbPassengers.SelectedValue;
96              int flightID = (int)cbFlights.SelectedValue;
97              decimal price = decimal.Parse(txtPrice.Text);
98              string seatNo = txtSeat.Text;
99              DateTime now = DateTime.Now;
100
101             // reading comboboxes
102             string ticketClass = ((ComboBoxItem)cbClass.SelectedItem).Content.ToString();
103             string checkInType = ((ComboBoxItem)cbCheckInType.SelectedItem).Content.ToString();
104
105             // Generating boarding code
106             DataRowView selectedFlight = (DataRowView)cbFlights.SelectedItem;
107             string flightInfo = selectedFlight["FlightInfo"].ToString();
108             string flightNo = flightInfo.Split('-')[0].Trim();
109
110             string boardingCode = $"{flightNo}-{seatNo}-{new Random().Next(1000, 9999)}";
111
112             Connection kon = new Connection();
113             using (SqlConnection conn = kon.createConnection())
114             {
115                 conn.Open();
```

```
117                 // Duplicates check
118                 string checkQuery = @"SELECT COUNT(*) FROM tbl_Booking B
119                                       INNER JOIN tbl_Ticket T ON B.booking_ID = T.booking_ID
120                                       WHERE B.passenger_ID = @passID AND T.flight_ID = @flightID";
121
122                 SqlCommand cmdCheck = new SqlCommand(checkQuery, conn);
123                 cmdCheck.Parameters.AddWithValue("@passID", passengerID);
124                 cmdCheck.Parameters.AddWithValue("@flightID", flightID);
125
126                 int count = (int)cmdCheck.ExecuteScalar();
127
128                 if (count > 0)
129                 {
130                     MessageBox.Show("This passenger already has a reservation for this flight!", "Duplicate Error", MessageBoxButton.OK, MessageBoxImage.Warning);
131                     return;
132                 }
133
134                 // 4. INSERT BOOKING
135                 string queryBooking = @"INSERT INTO tbl_Booking (bookingDate, status, totalPrice, passenger_ID)
136                                         VALUES (@date, 'Pending', @price, @passID);
137                                         SELECT SCOPE_IDENTITY();";
138
139                 SqlCommand cmdBooking = new SqlCommand(queryBooking, conn);
140                 cmdBooking.Parameters.AddWithValue("@date", now);
141                 cmdBooking.Parameters.AddWithValue("@price", price);
142                 cmdBooking.Parameters.AddWithValue("@passID", passengerID);
143
144                 int newBookingID = Convert.ToInt32(cmdBooking.ExecuteScalar());
145
146                 // 5. INSERT TICKET
147                 string queryTicket = @"INSERT INTO tbl_Ticket (seatNo, class, price, flight_ID, booking_ID)
148                                        VALUES (@seat, @class, @price, @flightID, @bookingID);
149                                        SELECT SCOPE_IDENTITY();";
150
151                 SqlCommand cmdTicket = new SqlCommand(queryTicket, conn);
152                 cmdTicket.Parameters.AddWithValue("@seat", seatNo);
153                 cmdTicket.Parameters.AddWithValue("@class", ticketClass);
154                 cmdTicket.Parameters.AddWithValue("@price", price);
```



```
                    SqlCommand cmdTicket = new SqlCommand(queryTicket, conn);
                    cmdTicket.Parameters.AddWithValue("@seat", seatNo);
                    cmdTicket.Parameters.AddWithValue("@class", ticketClass);
                    cmdTicket.Parameters.AddWithValue("@price", price);
                    cmdTicket.Parameters.AddWithValue("@flightID", flightID);
                    cmdTicket.Parameters.AddWithValue("@bookingID", newBookingID);

                    int newTicketID = Convert.ToInt32(cmdTicket.ExecuteScalar());

                    // 6. INSERT CHECK-IN
                    string queryCheckIn = @"INSERT INTO tbl_CheckIn (checkinTime, type, boardingPassCode, ticket_ID, desk_ID)
                                VALUES (@time, @type, @code, @ticketID, 'AdminDesk')";

                    SqlCommand cmdCheckIn = new SqlCommand(queryCheckIn, conn);
                    cmdCheckIn.Parameters.AddWithValue("@time", now);
                    cmdCheckIn.Parameters.AddWithValue("@type", checkInType);
                    cmdCheckIn.Parameters.AddWithValue("@code", boardingCode);
                    cmdCheckIn.Parameters.AddWithValue("@ticketID", newTicketID);

                    cmdCheckIn.ExecuteNonQuery();

                    MessageBox.Show("Reservation Created Successfully!", "Success", MessageBoxButton.OK, MessageBoxImage.Information);
                    this.Close();
                }
            }
            catch (Exception ex)
            {
                MessageBox.Show("Error creating reservation: " + ex.Message);
            }
        }

        1 reference
        private void btnCancel_Click(object sender, RoutedEventArgs e)
        {
            this.Close();
        }
    }
```

3.12 btnSave_Click

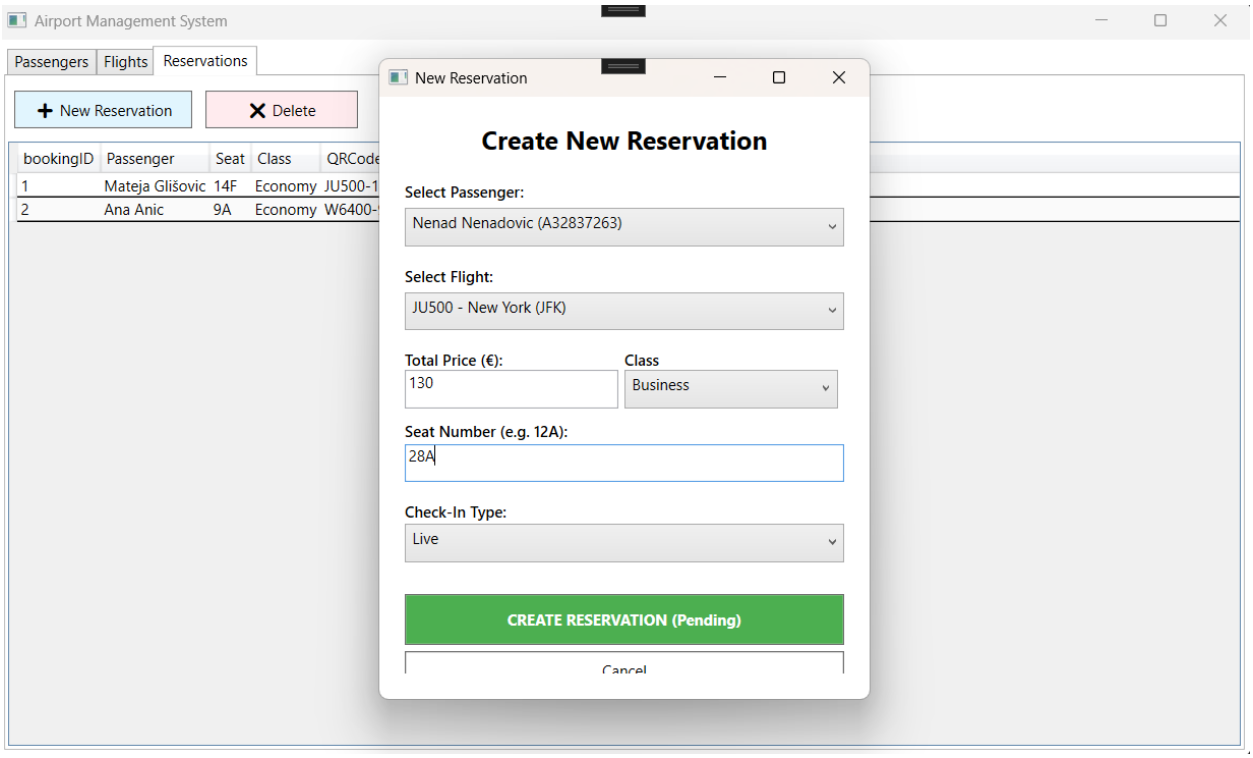
3.8 Putnički Dashboard: Dinamički UI i Stanja

Putnički deo aplikacije fokusiran je na preglednost i interakciju. Korišćena je napredna logika za manipulaciju UI elementima (User Interface) na osnovu statusa podataka u bazi.

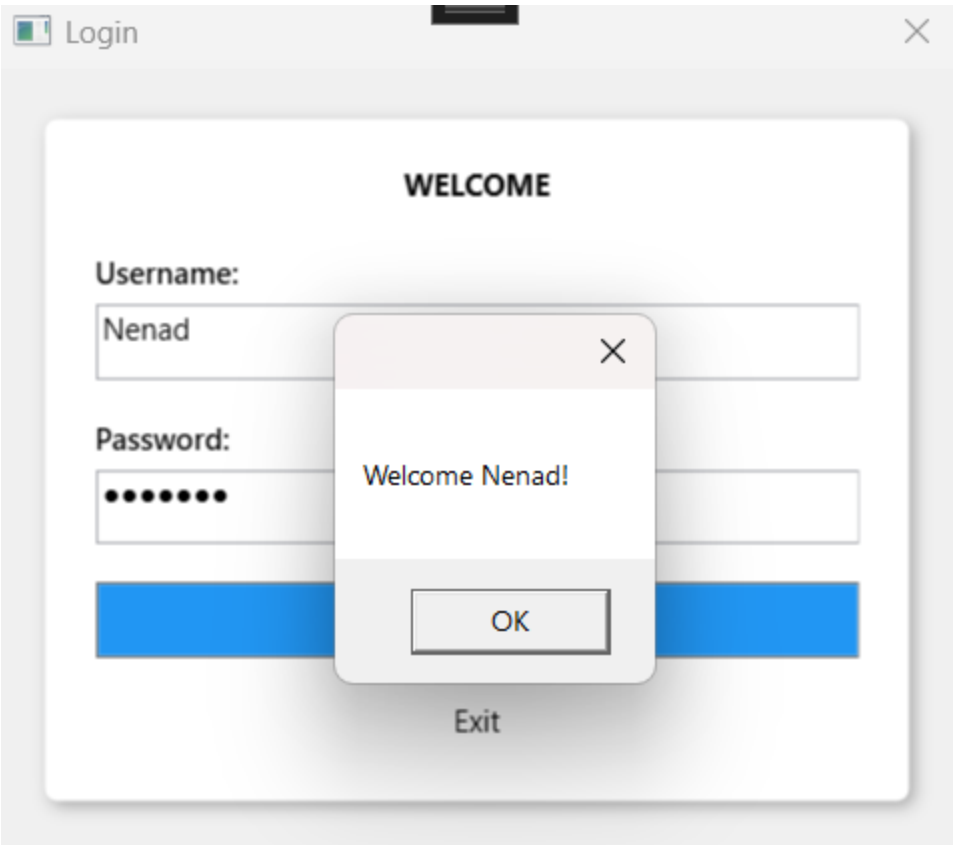
Logika potvrde leta: Aplikacija koristi događaj `SelectionChanged` na tabeli letova.

- Ako je status leta **"Pending"**: Dugme za potvrdu postaje zeleno i omogućava se interakcija. Prikazuje se CheckBox za prihvatanje uslova korišćenja.
- Ako je status leta **"Confirmed"**: Dugme postaje sivo i onemogućava se ponovno plaćanje.

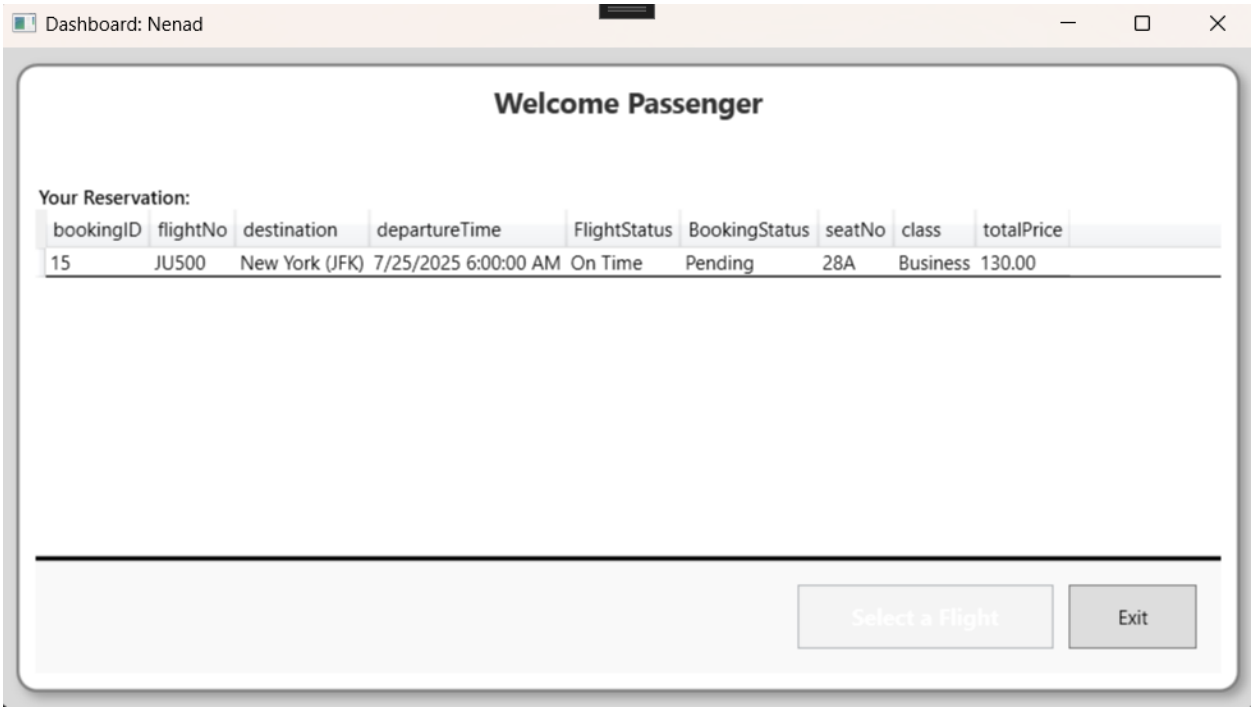
Ovo je primer **"State-Based"** (baziranog na stanju) dizajna, gde softver vodi korisnika kroz proces i sprečava nedozvoljene akcije (npr. plaćanje već plaćenog leta).



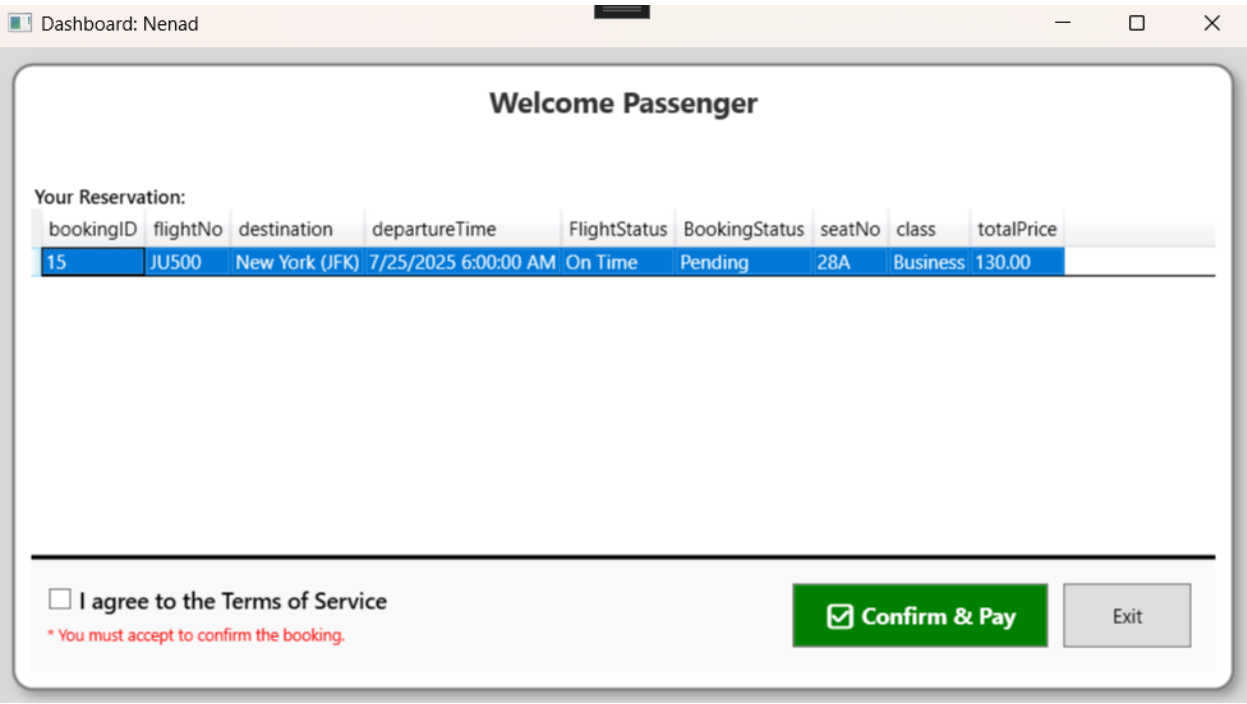
3.13 Creating Reservation (pending)



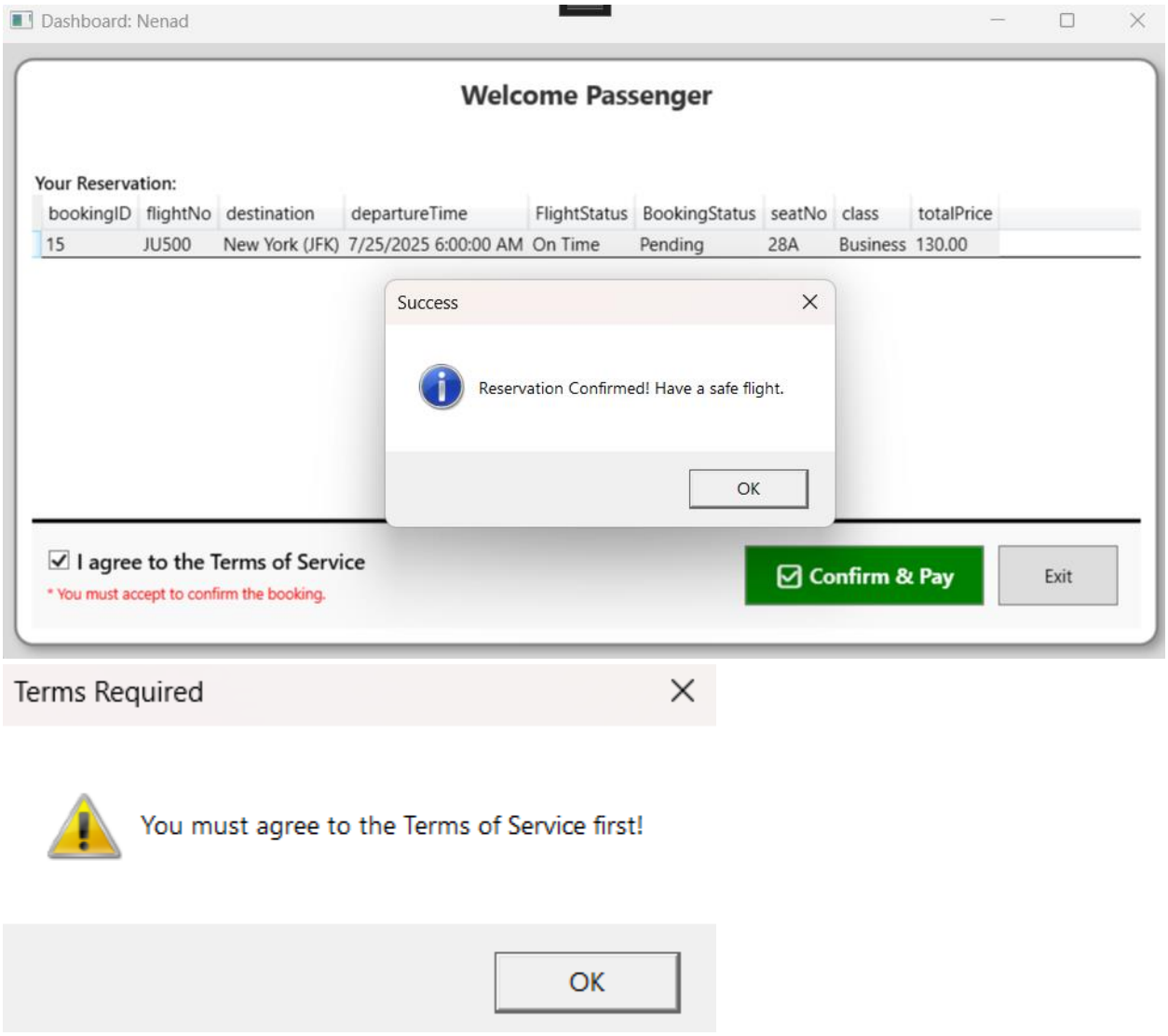
3.14 LoginWindow.xaml (passenger)



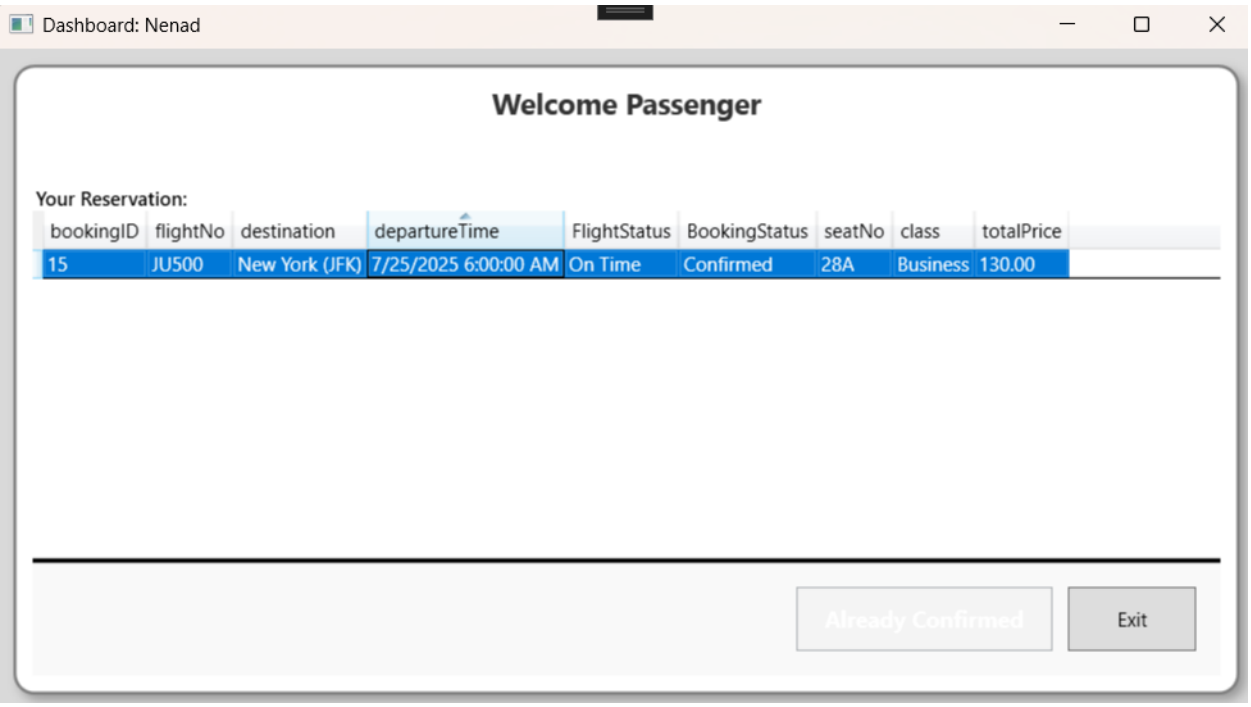
3.15 PassengerDashboard.xaml (not selected)



3.16 PassengerDashboard.xaml (selected)



3.17 PassengerDashboard.xaml (checkbox)



3.18 PassengerDashboard.xaml (confirmed)

```
76      private void dgMyFlights_SelectionChanged(object sender, SelectionChangedEventArgs e)
77      {
78          // if nothing loaded, reset the button
79          if (dgMyFlights.SelectedItem == null)
80          {
81              ResetUI();
82              return;
83          }
84
85          DataRowView row = (DataRowView)dgMyFlights.SelectedItem;
86          string bookingStatus = row["BookingStatus"].ToString().Trim();
87
88          if (bookingStatus == "Pending")
89          {
90              // button to confirm
91              btnAction.Content = "☑ Confirm & Pay";
92              btnAction.Background = new SolidColorBrush(Colors.Green);
93              btnAction.IsEnabled = true;
94
95              // show checkbox
96              pnlTerms.Visibility = Visibility.Visible;
97              cbTerms.IsChecked = false;
98          }
99          else if (bookingStatus == "Confirmed")
100         {
101             // if its paid
102             btnAction.Content = "Already Confirmed";
103             btnAction.Background = new SolidColorBrush(Colors.Gray);
104             btnAction.IsEnabled = false;
104                 btnAction.IsEnabled = false;
105
106                 pnlTerms.Visibility = Visibility.Hidden;
107             }
108             else
109             {
110                 // if its canceled
111                 btnAction.Content = bookingStatus;
112                 btnAction.IsEnabled = false;
113                 pnlTerms.Visibility = Visibility.Hidden;
114             }
115         }
116
            1 reference
117         private void ResetUI()
118         {
119             btnAction.Content = "Select a Flight";
120             btnAction.Background = new SolidColorBrush(Colors.Gray);
121             btnAction.IsEnabled = false;
122             pnlTerms.Visibility = Visibility.Hidden;
123         }
124
            1 reference
125         private void btnAction_Click(object sender, RoutedEventArgs e)
126         {
127             // more protection
128             if (dgMyFlights.SelectedItem == null) return;
```

```
// more protection
if (dgMyFlights.SelectedItem == null) return;

// checking if its ckecked (how original)
if (cbTerms.IsChecked == false)
{
    MessageBox.Show("You must agree to the Terms of Service first!", "Terms Required", MessageBoxButton.OK, MessageBoxImage.Warning);
    return;
}

try
{
    DataRowView row = (DataRowView)dgMyFlights.SelectedItem;
    int bookingID = (int)row["booking_ID"];

    Connection kon = new Connection();
    using (SqlConnection conn = kon.createConnection())
    {
        conn.Open();

        // UPDATE STATUS NA CONFIRMED
        string query = "UPDATE tbl_Booking SET status = 'Confirmed' WHERE booking_ID = @id";
        SqlCommand cmd = new SqlCommand(query, conn);
        cmd.Parameters.AddWithValue("@id", bookingID);
        cmd.ExecuteNonQuery();

        MessageBox.Show("Reservation Confirmed! Have a safe flight.", "Success", MessageBoxButton.OK, MessageBoxImage.Information);

        LoadMyFlights();
    }
```

```
            catch (Exception ex)
            {
                MessageBox.Show("Error: " + ex.Message);
            }
        }

        1 reference
        private void btnExit_Click(object sender, RoutedEventArgs e)
        {
            LoginWindow login = new LoginWindow();
            login.Show();
            this.Close();
        }
    }
}
```

3.19 dgMyFlights_SelectionChanged

3.9 Zaštita od SQL Injection napada

Kroz celu aplikaciju striktno su korišćeni **parametrizovani upiti** (`SqlParameter`). Umesto direktnog lepljenja stringova (konkatenacije), vrednosti se prosleđuju kao parametri (npr. `@username`, `@price`).

Ovaj pristup onemogućava zlonamerne korisnike da umetanjem SQL koda manipulišu bazom podataka, što je standard u razvoju bezbednog softvera.